DATA STRUCUTRES AND ALGORITHMS

23CSE203

LAB MANUAL

Submitted by	
Name	MUTTABATHULA HANSINI SARORA
Roll No	AV.SC.U4CSE25041
Year/Sem/Section	2nd year/Sem-3/CSE-A
Credits	05

Submitted to	
Name	Dr. RAJ KUMAR BATCHU
Department	School of Computing
Designation	Asst. Professor (SI Grade)

S.NO	TITLE	PAGE NO	DATE
1.	**WEEK 1:** 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	5 – 13	23-07-26
2.	**WEEK 2:** 2a. Write a python program to perform linear search on the give input. 2b. Write a python program to perform binary search on the given. Verify by taking sorted array as an input. 2c. Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.	14 – 20	30-07-26
3.	**WEEK 3:** 3a. Write a program to perform Bubble Sort technique on the user given input. 3b. Write a program to perform Insertion Sort technique on the user given input. 3c. Write a program to perform Selection Sort technique on the user given input. 3d. Write a program to perform Merge Sort technique on the user given input. 3e. Write a program to perform Quick Sort technique on the user given input.	21- 32	06-08-26
4.	**WEEK 4:** 4a. Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user. a. Create a linked list.	33 - 40	13-08-26

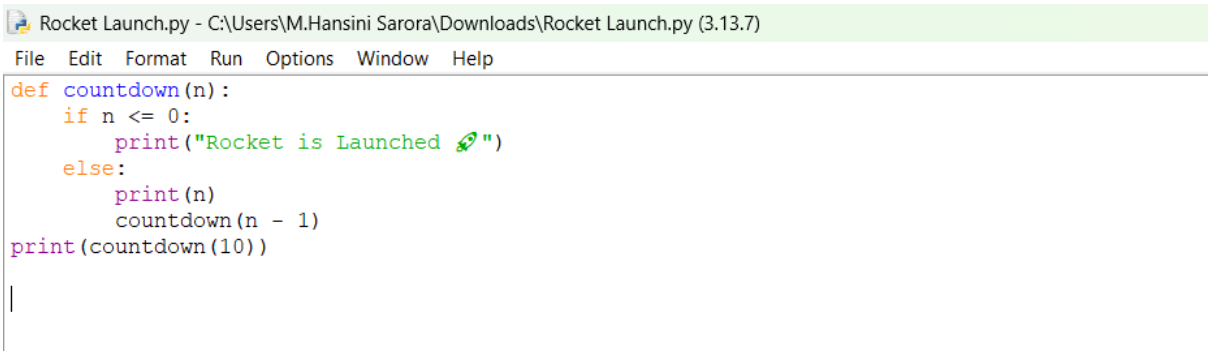
4.	b. Insert at beginning. c. Insert at end. d. Insert at a specific index. e. Delete by value. f. Delete 1st node. g. Delete last node. h. Count no. of nodes i. Display. j. Exit. 4b. Write a python program to implement a double linked list which would perform the below operations, by taking an input from the user. 4c. Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user. Also add these two options accordingly. 1. Display head and tail. 2. Print data from tail to head.	40 – 62	13-08-26
5.	**WEEK 5:** Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.	63 - 69	27-08-26
6.	**WEEK 6:** To implement Queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu driven approach. 6a. Using Array, Linked list, Data structures. 6b. Circular Queue.	70 - 80	03-09-26
7.	**WEEK 7:** Write a python program to implement binary tree 7a. Using Array, and 7b. Linked List With all of its operation.	81 - 92	24-09-26

8.	**WEEK 8:** Write a python program to implement Binary Search tree with its operation.	93 - 98	08-10-26
9.	**WEEK 9:** Write a python program to implement AVL trees with its operation.	99 – 104	22-10-26
10.	**WEEK 10:** Write a python program to implement hashing using separating chaining, linear, and quadratic probing.	105 - 110	29-10-26

WEEK – 1

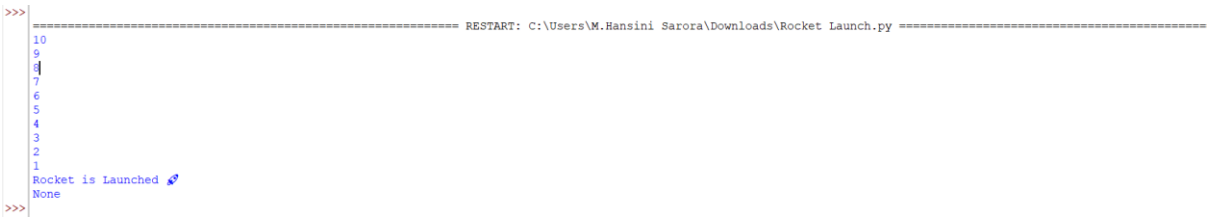
1a. AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursive. Write a recursive python program to display a countdown from n to 1, followed by "Launch".

PROGRAM:

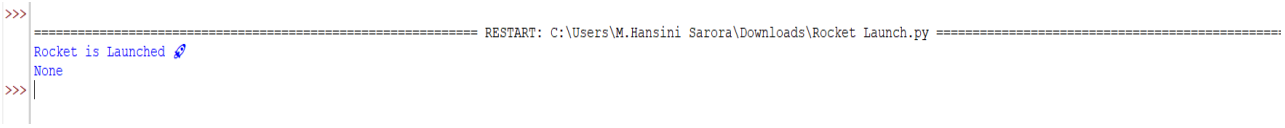


OUTPUT:

Positive Case;



Negative Case;



EXPLANATION:

Condition 1: Positive Input (n > 0)

- The function checks if n <= 0. Since n is positive, this condition is False.
- It executes the else block:
  - Prints the current value of n.
  - Calls itself recursively with n - 1.
- This repeats until n becomes 0.
- Finally, when n = 0, the condition n <= 0 becomes True.
- After the recursive calls finish, the function returns None (since no return is defined).
- The calling print (countdown (10)) will display the countdown, the launch message, and then None.

Condition 2: Negative Input (n <= 0)

- The condition if n <= 0 is immediately **True**.
- It directly prints:

  Rocket is Launched 🚀
- No recursion happens because the else block is skipped.
- The function ends and returns None.
- The calling print (countdown (-1)) will display:

  Rocket is Launched 🚀

  None

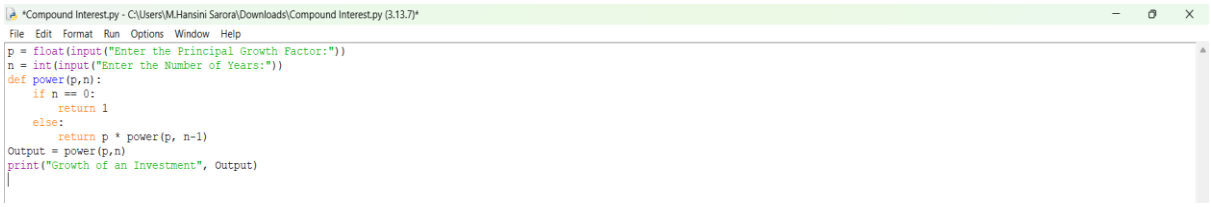
ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Syntax Error Forgot to give ":" in the control statement.	Later it was rectified by giving the ":"

2.	Type Error I have entered a non – integer.	It was rectified by passing only the integers.

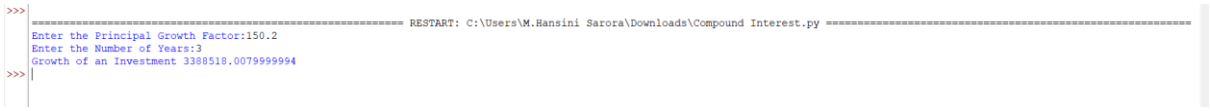
1b. AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power), Where p = principal growth factor, n = number of years.

PROGRAM:

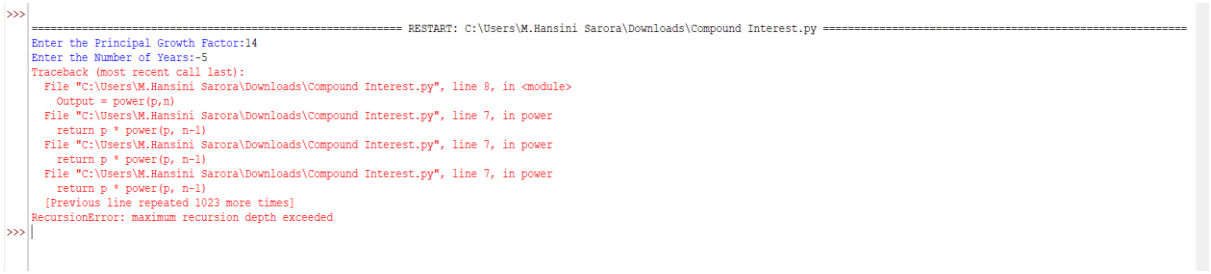


OUTPUT:

Positive Case;



Negative Case;



EXPLANATION:

Condition 1: Positive Input (n ≥ 0)

- User enters a positive number of years (e.g., n = 3).
- The function checks if n == 0. Since n is positive, this is False.

- It executes the else block:
    - Multiplies p by the result of power (p, n-1).
- Recursion continues until n becomes 0.
- At n = 0, the base case returns 1.
- The recursive calls unwind, multiplying p repeatedly.
- The output is p^n, i.e., the growth factor raised to the number of years.

Condition 2: Negative Input (n < 0)

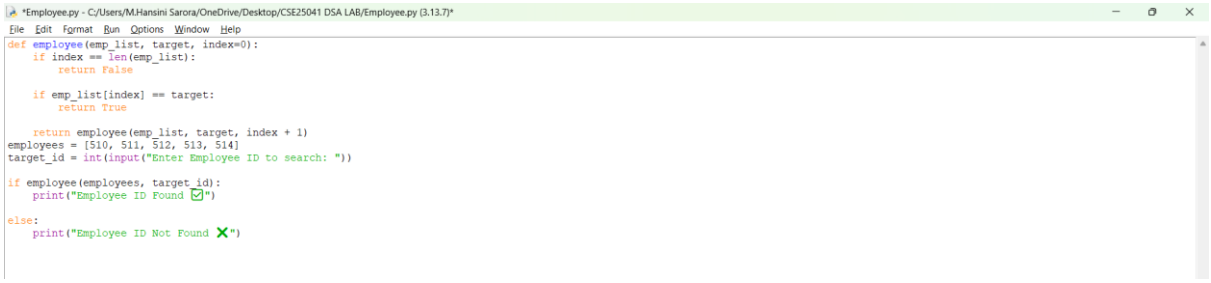
- User enters a negative number of years (e.g., n = -2).
- The function checks if n == 0. Since n is negative, this is **False**.
- It executes the else block:
    - Calls power (p, n-1) with an even smaller negative number.
- Recursion continues indefinitely (-2 → -3 → -4 → …).
- The base case (n == 0) is never reached.
- Python raises a "Recursion Error: maximum recursion depth exceeded."

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Floating-Point I have entered a floating-point value in the years.	Later it was rectified by giving the integer value.
2.	Large Input Error I have given a huge input value.	It was rectified by passing default limit of Python.

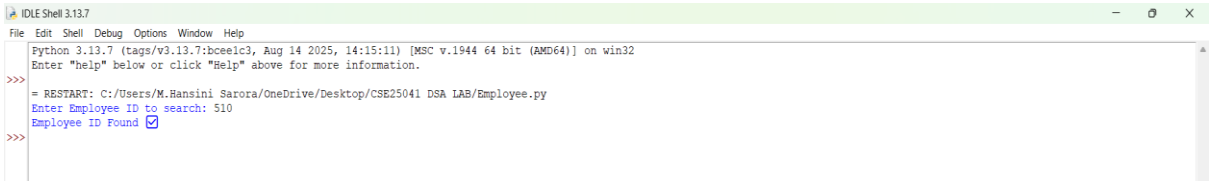
1c. AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

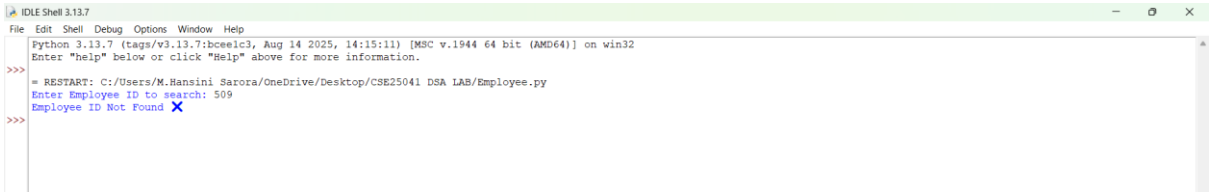


OUTPUT:

Positive Case;



Negative Case;



EXPLANATION:

Condition 1: Employee ID Found

- The function starts at index 0.
- It compares emp_list[index] with the target.
- If a match is found, the function immediately returns True.

- The recursion stops early — no need to check further elements.
- The main program prints:

Employee ID Found ✅

Condition 2: Employee ID Not Found

- The function checks each element recursively.
- If the end of the list is reached (index == len(emp_list)), the base case returns False.
- The recursion unwinds, propagating False back to the main program.
- The main program prints:

Employee ID Not Found ❌

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Empty List I haven't entered the values in the list.	The empty list was filled with the emp IDs.
2.	Logical Error I have entered the duplicate values.	It was rectified by removing the duplicates.

1d. AIM: In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
*Factorial.py - C:\Users\M.Hansini Sarora\Downloads\Factorial.py (3.13.7)*
File  Edit  Format  Run  Options  Window  Help
n = int(input("Enter the Number:"))
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
print(f"Possible ways to arrange {n} different parcels is",factorial(n))
```

OUTPUT:

Positive Case;

```
>>>
========== RESTART: C:\Users\M.Hansini Sarora\Downloads\Factorial.py ==========
Enter the Number:6
Possible ways to arrange 6 different parcels is 720
>>>
```

Negative Case;

```
>>>
========== RESTART: C:\Users\M.Hansini Sarora\Downloads\Factorial.py ==========
Enter the Number:-58
Traceback (most recent call last):
  File "C:\Users\M.Hansini Sarora\Downloads\Factorial.py", line 7, in <module>
    print(f"Possible ways to arrange {n} different parcels is",factorial(n))
  File "C:\Users\M.Hansini Sarora\Downloads\Factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  File "C:\Users\M.Hansini Sarora\Downloads\Factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  File "C:\Users\M.Hansini Sarora\Downloads\Factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  [Previous line repeated 1023 more times]
RecursionError: maximum recursion depth exceeded
>>>
```

EXPLANATION:

Condition 1: When n == 0

- The function checks:

  if n == 0:

  return 1

- By definition, 0! = 1.
- So if the user enters 0, the program outputs:

  Possible ways to arrange 0 different parcels is 1

Condition 2: When n > 0

- The function executes:

  else:

  return n * factorial(n - 1)

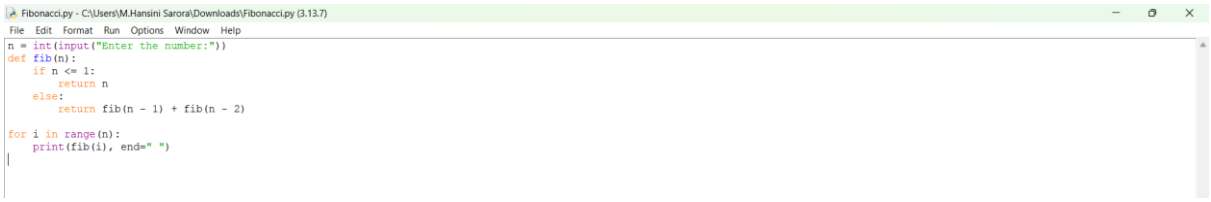
- This means the function calls itself with a smaller number until it eventually reaches n == 0.
- Uses recursion to multiply numbers down to 1.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Negative Input I haven't entered the negative value.	Later, It was rectified by entering positive values.
2.	Large Input Error I have entered a very large number.	It was rectified by entering limited values.

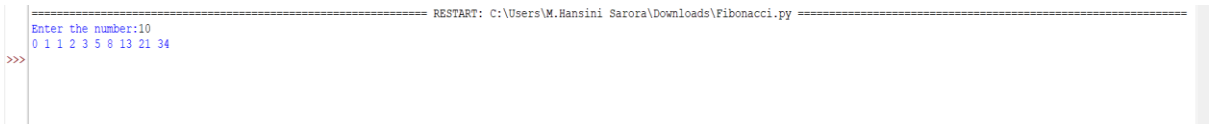
1e. AIM: Write a recursive python program to print the first n yerms of the Fibonacci Series.

PROGRAM:

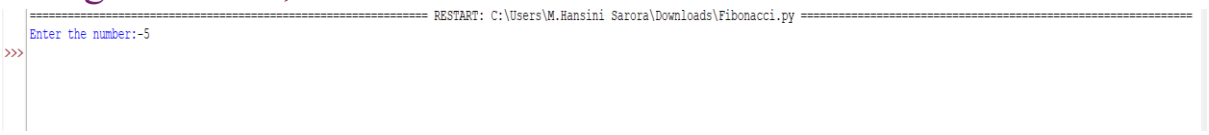


OUTPUT:

Positive Case;



Negative Case;



EXPLANATION:

Condition 1: When n <= 1

- Inside the function:

```
if n <= 1:
    return n
```

- This is the **base case** of recursion.
- If n = 0, it returns 0.
- If n = 1, it returns 1.
- These are the starting points of the Fibonacci sequence..

Condition 2: When n > 1

- The function executes:

python

```
else:
    return fib(n - 1) + fib(n - 2)
```

- This means the Fibonacci number at position n is the **sum of the two previous numbers**.
- The recursion keeps breaking down until it reaches the base case (n <= 1).

ERROR TABLE:

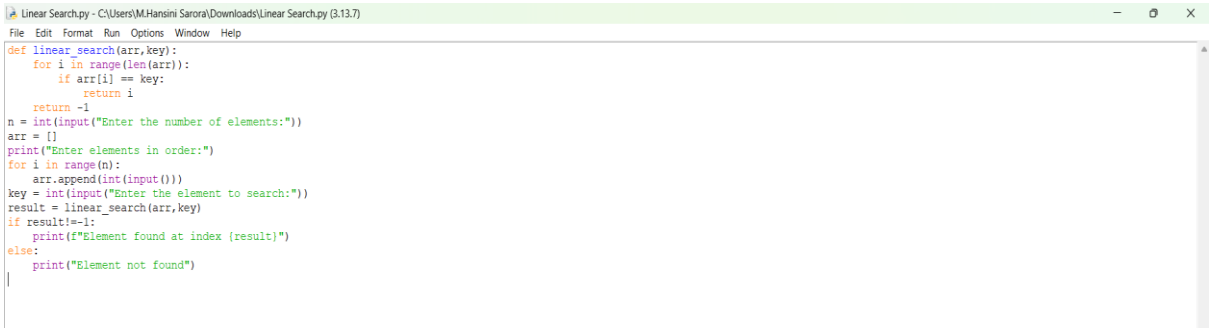
S.NO	Error Obtained	Rectification

1.	Negative Input I haven't entered the negative value.	Later, it was rectified by entering positive values.
2.	Very large input Recursion depth exceeded.	It was rectified by entering limited values.

WEEK – 2

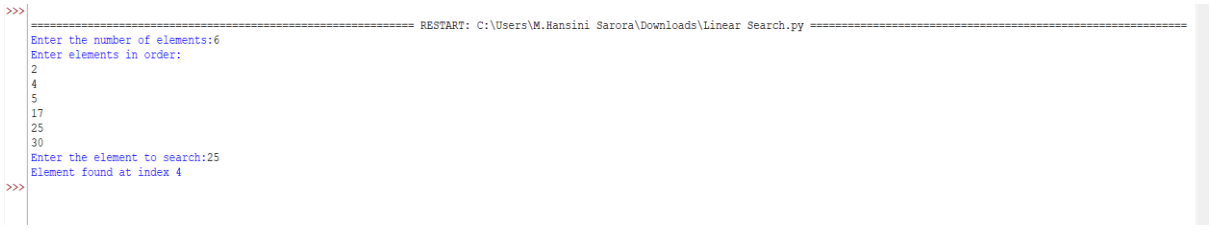
2a. AIM: Write a python program to perform linear search on the give input.

PROGRAM:



OUTPUT:

Positive Case;



Negative Case;

```
>>>
========= RESTART: C:\Users\M.Hansini Sarora\Downloads\Linear Search.py ========
Enter the number of elements:5
Enter elements in order:
2
8
10.2
Traceback (most recent call last):
  File "C:\Users\M.Hansini Sarora\Downloads\Linear Search.py", line 10, in <module>
    arr.append(int(input()))
ValueError: invalid literal for int() with base 10: '10.2'
>>>
```

EXPLANATION:

Condition 1: Element Found

- The program takes an array of numbers from the user.
- It then asks for a key (the element to search).
- The linear_search function goes through the array one element at a time.
- If the current element matches the key, the function returns its index.
- In this case, the program prints:

  Element found at index <position>

Example:

- Input array: [10, 20, 30, 40]
- Key: 30
- Output: Element found at index 2

Condition 2: Element Not Found

- If the loop finishes and no element matches the key, the function returns -1.
- The program then prints:

  Element not found

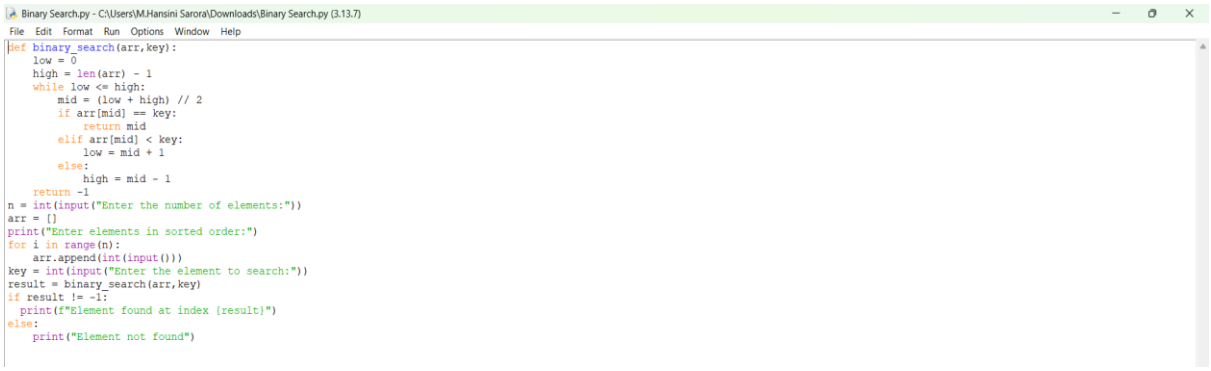
ERROR TABLE:

S.NO	Error Obtained	Rectification

1.	Indentation error Return -1 was typed inside the if loop condition.	Later, it was rectified by typing it outside the if loop and inside the for loop.
2.	Syntax error Invalid syntax, ! == -1	It was rectified by re-typing, ! = -1.

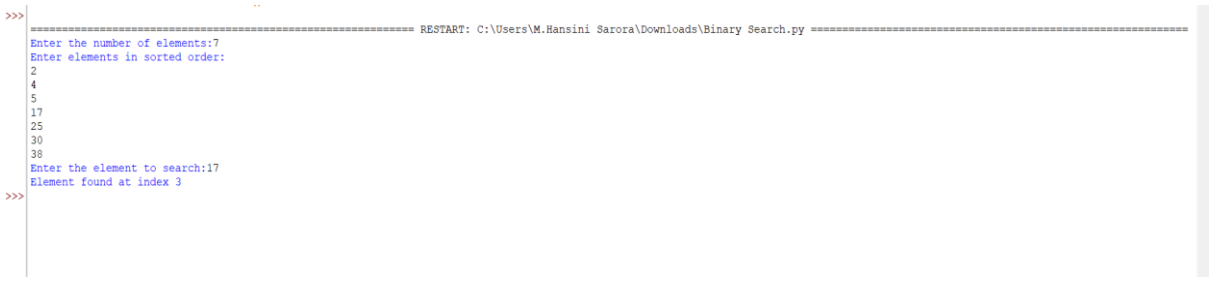
2b. AIM: Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

PROGRAM:



OUTPUT:

Positive Case;



Negative Case;

```
>>>
========== RESTART: C:\Users\M.Hansini Sarora\Downloads\Binary Search.py ==========
Enter the number of elements:7
Enter elements in sorted order:
2
6
75
DSA
Traceback (most recent call last):
  File "C:\Users\M.Hansini Sarora\Downloads\Binary Search.py", line 17, in <module>
    arr.append(int(input()))
ValueError: invalid literal for int() with base 10: 'DSA'
>>>
```

EXPLANATION:

Condition 1: Element Found

- The program asks the user to enter elements in **sorted order** (binary search only works on sorted arrays).
- The binary_search function starts with two pointers:
  - low → beginning of the array
  - high → end of the array
- It calculates the middle index mid = (low + high) // 2.
- If the middle element equals the key, the function returns that index.
- The program then prints:

  Element found at index <position>

Example:

- Input array: [5, 10, 15, 20, 25]
- Key: 20
- Output: Element found at index 3

Condition 2: Element Not Found

- If the key is not equal to the middle element, the algorithm adjusts the search range:
  - If arr[mid] < key, search continues in the **right half** (low = mid + 1).
  - If arr[mid] > key, search continues in the **left half** (high = mid - 1).
- This process repeats until low > high.

- If the key is not found, the function returns -1.
- The program then prints:

  Element not found

Example:

- Input array: [5, 10, 15, 20, 25]
- Key: 30
- Output: Element not found

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Incorrect loop condition low < high	Later, it was rectified by writing as low <= high.
2.	Wrong middle calculation mid = (low + high) / 2	It was rectified by re-typing, mid = (low + high) // 2.

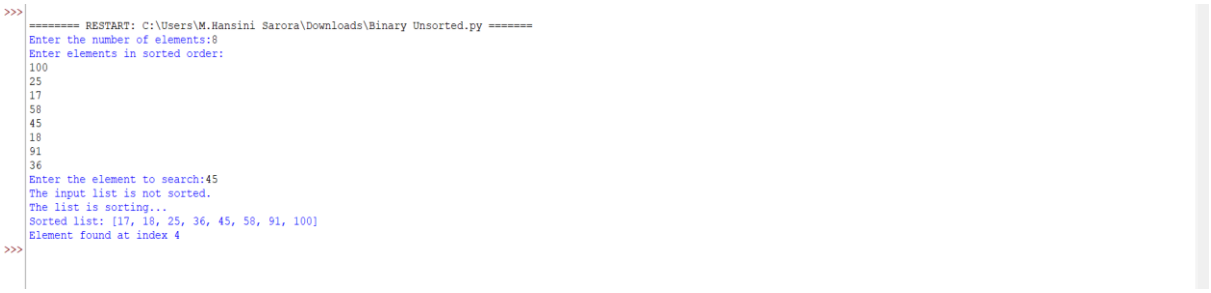
2c. AIM: Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.

PROGRAM:



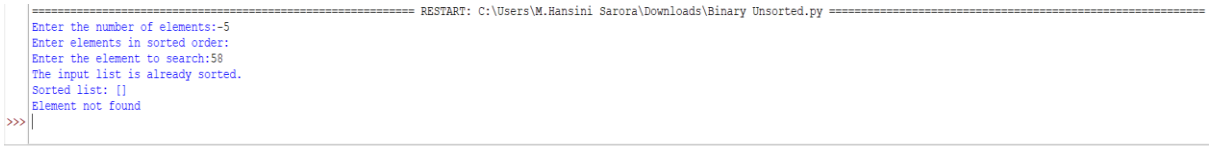
OUTPUT:

Positive Case;



Negative Case;

A semantic error occurs in this case,



EXPLANATION:

Condition 1: Input List Already Sorted

- The program first checks if the entered list is already sorted or not.
- If true, it prints:

The input list is already sorted.

- Then it directly applies **binary search** on the given list.
- If the key is found, it prints the index:

  Element found at index <position>
- Otherwise, it prints:

  Element not found

  Example:
- Input array: [5, 10, 15, 20]
- Key: 15
- Output:

  The input list is already sorted.

  Sorted list: [5, 10, 15, 20]

  Element found at index 2

Condition 2: Input List Not Sorted

- If the entered list is not sorted, the program prints

  The input list is not sorted.

  The list is sorting...
- It then sorts the list using:

  arr.sort()
- After sorting, it prints the sorted list and applies **binary search**.
- If the key is found, it prints the index in the **sorted list**.
- Otherwise, it prints:

  Element not found

Example:

- Input array: [20, 5, 15, 10]
- Key: 10
- Output:

  The input list is not sorted.

  The list is sorting...

Sorted list: [5, 10, 15, 20]

Element found at index 1

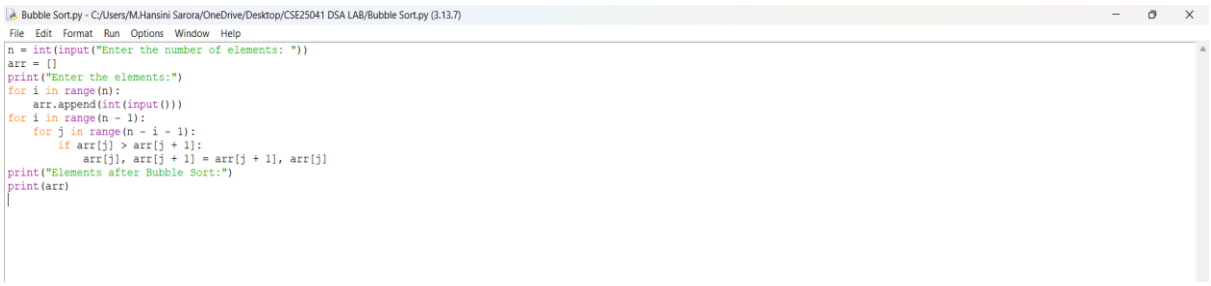
ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Duplicate elements It does not guarantee the 1st or last occurrence	Later, it was rectified by not entering the duplicate values.
2.	Semantic error Given an empty list	It was rectified by entering the values in the list.

WEEK – 3

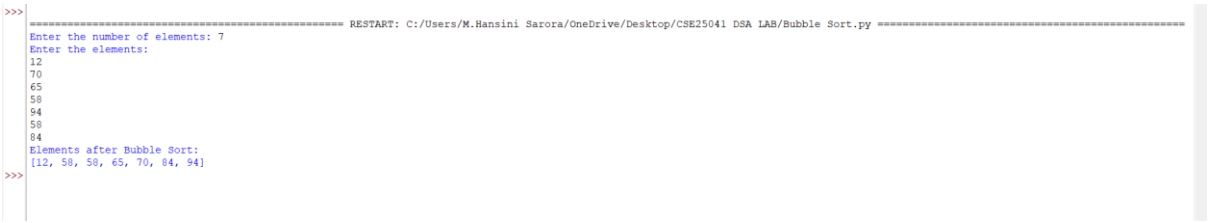
3a. AIM: Write a program to perform Bubble Sort technique on the user given input.

PROGRAM:

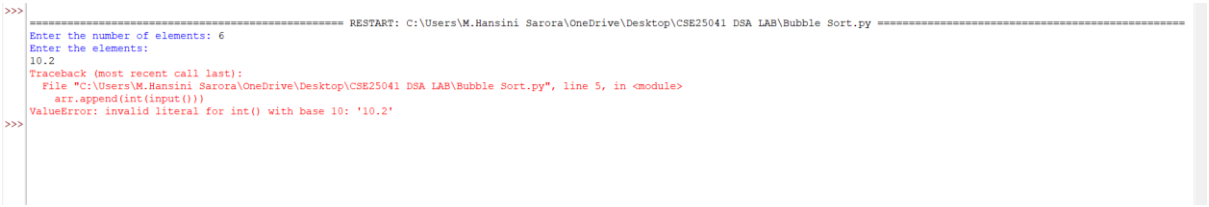


OUTPUT:

Positive Case;



Negative Case;



EXPLANATION:

1. Start by taking input size,

The program first asks how many elements (n) the user wants to sort. This sets the boundary for loops.

2. Input the elements into the list,

The user enters n numbers, which are stored in the list arr.

3. Begin Bubble Sort process,

Sorting starts with two nested loops:

- Outer loop (i) → Controls the number of passes. Runs n-1 times because after each pass one element is placed correctly at the end.
- Inner loop (j) → Compares adjacent elements in each pass.

4. Condition check inside inner loop,

- The condition is: if the current element is greater than the next element (arr[j] > arr[j+1]).
- If this condition is true, the two elements are swapped.
- If the condition is false, no swap is made and the loop continues.

5. Effect of condition and swapping,

- Each time the condition is true, larger elements move step by step toward the end of the list.
- After the first pass, the largest element is at the last position.
- After the second pass, the second-largest is in its correct position, and so on.

6. Completion of sorting,

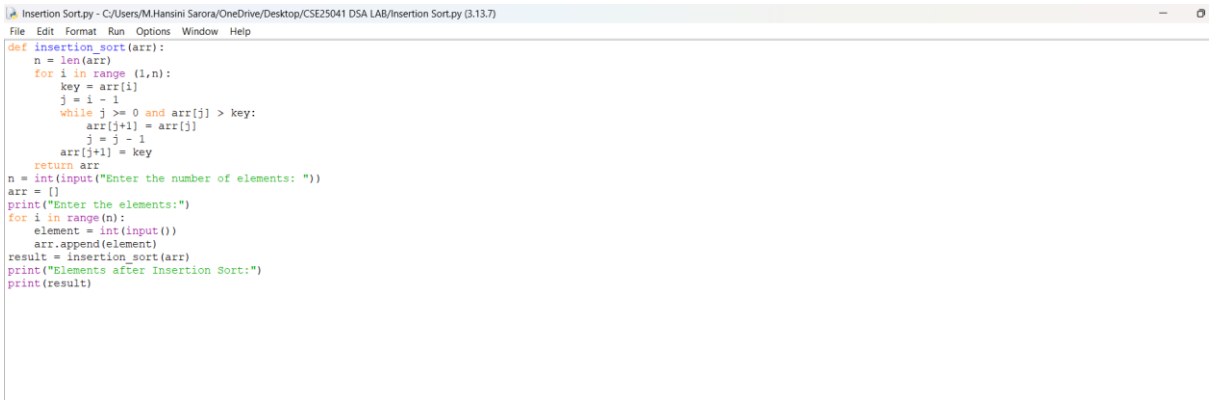
When all passes are finished, the list is sorted in ascending order. The program then prints the sorted list.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Logical Error if arr[j] > arr[j + 1]:	It will sort in descending order, so it was rectified by changing it to "<" this.
2.	Semantic error Given an empty list	It was rectified by entering the values in the list.

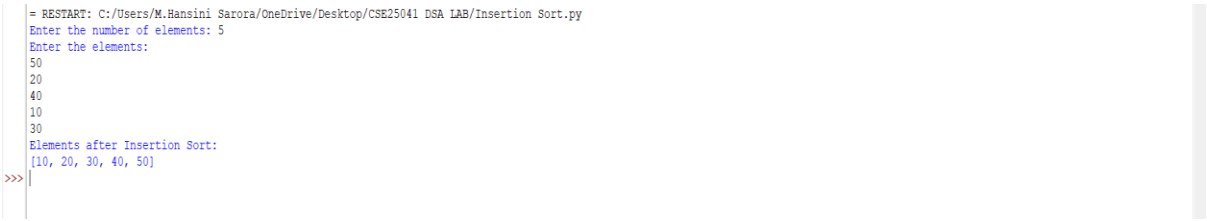
3b. AIM: Write a program to perform Insertion Sort technique on the user given input.

PROGRAM:

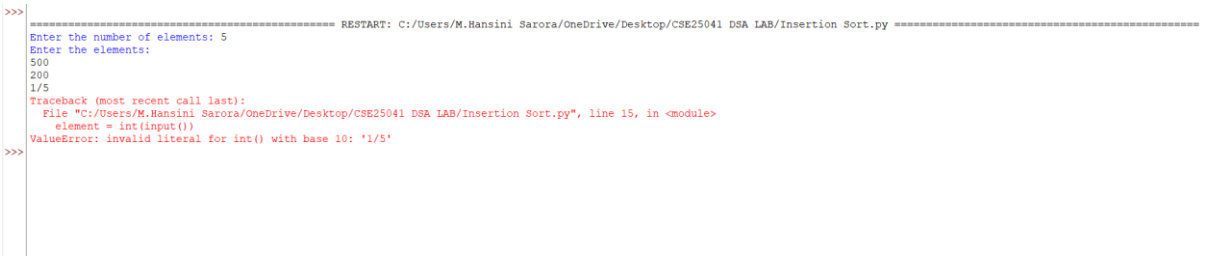


OUTPUT:

Positive Case;



Negative Case;



EXPLANATION:

1. Input Phase,

- The program first asks the user for the number of elements (n).
- Then it takes n integer inputs one by one and stores them in the list arr.

2. Insertion Sort Process,

Insertion Sort works by building the sorted list one element at a time:

- Outer loop (i from 1 to n-1): Starts from the second element because the first element is trivially sorted. Each iteration picks the current element (key) and tries to place it in the correct position among the already sorted part of the list.
- **Condition check inside while loop:**
  - The condition is: while j >= 0 and arr[j] > key
  - Meaning: As long as the previous element (arr[j]) is greater than the current element (key), shift it one position to the right.
  - If the condition is true, shifting happens.

- If the condition is false, the correct position for key is found.

- **Placement of key:** After shifting larger elements, the key is placed at its correct position (arr[j+1] = key).

3. **Output Phase,**

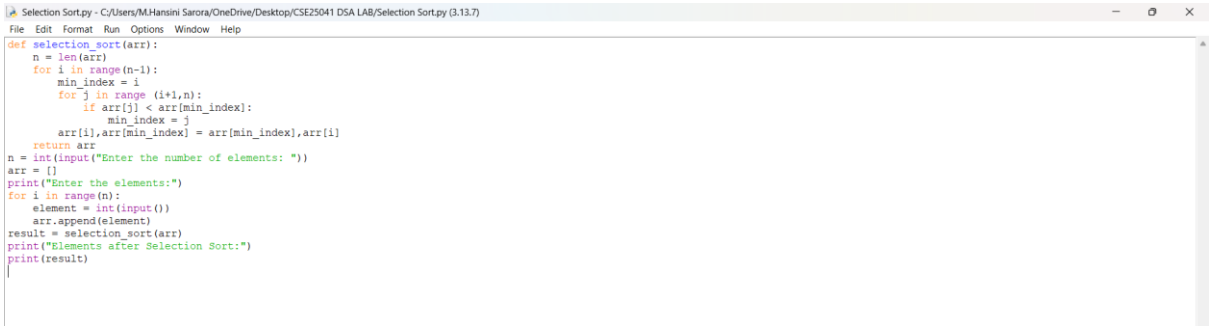
- After all passes, the list is fully sorted in ascending order.
- The program prints the sorted list.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Syntax error	Later, it was rectified by typing it as ':' in loop.
2.	Condition check j <= 0	It was rectified by correcting it with '>'. It could cause negative indexing errors.

3c. AIM: Write a program to perform Selection Sort technique on the user given input.

PROGRAM:



OUTPUT:

Positive Case;

```
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Selection Sort.py
Enter the number of elements: 5
Enter the elements:
12
11
13
5
6
Elements after Selection Sort:
[5, 6, 11, 12, 13]
>>>
```

Negative Case;

```
>>>
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Selection Sort.py ==========
Enter the number of elements: 5
Enter the elements:
12
1
abc
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Selection Sort.py", line 14, in <module>
    element = int(input())
ValueError: invalid literal for int() with base 10: 'abc'
>>>
```

EXPLANATION:

1. Input Phase

- The program first asks the user for the number of elements (n).
- Then it takes n integer inputs one by one and stores them in the list arr.

2. Selection Sort Process

Selection Sort works by repeatedly finding the smallest element from the unsorted part of the list and placing it at the beginning.

- **Outer loop (i from 0 to n-2):** Each iteration selects the position where the next smallest element should be placed.
- **Initialize minimum index:** At the start of each pass, min_index = i. This assumes the current element is the smallest.
- **Inner loop (j from i+1 to n-1):** Compares the current minimum element with the rest of the unsorted elements.
    - **Condition check:** if arr[j] < arr[min_index]

- If **true**, update min_index = j (a new smaller element is found).
- If **false**, keep the current min_index.
- **Swap after inner loop:** After checking all elements, swap the smallest element found (arr[min_index]) with the element at position i. This places the correct element in its sorted position.

3. Output Phase

- After all passes, the list is fully sorted in ascending order.
- The program prints the sorted list.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Syntax error	Later, it was rectified by typing it as ':' in loop.
2.	Meaningless behaviour when n = 0 or negative	Add a condition: if n <= 0, print "No elements to sort" and skip sorting.

3d. AIM: Write a program to perform Merge Sort technique on the user given input.

PROGRAM:

```
*Merge Sort.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Merge Sort.py (3.13.7)*
File  Edit  Format  Run  Options  Window  Help
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    i = j = k = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1

    return arr
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    element = int(input())
    arr.append(element)

result = merge_sort(arr)
print("Elements after Merge Sort:")
print(result)
```

OUTPUT:

Positive Case;

```
>>> ========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Merge Sort.py ==========
Enter the number of elements: 5
Enter the elements:
64
25
12
22
11
Elements after Merge Sort:
[11, 12, 22, 25, 64]
>>>
```

Negative Case;

```
>>>
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Merge Sort.py ==========
Enter the number of elements: 5
Enter the elements:
64
25
12.25
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Merge Sort.py", line 36, in <module>
    element = int(input())
ValueError: invalid literal for int() with base 10: '12.25'
>>>
```

EXPLANATION:

Step 1: Input Phase

- The program first asks the user for the number of elements (n).
- Then it takes n integer inputs one by one and stores them in the list arr.

Step 2: Merge Sort Process

Merge Sort is a divide and conquer algorithm. It divides the list into smaller parts, sorts them individually, and then merges them back together.

Condition 1: Base Case

- If the list has one or zero elements, it is already sorted.
    - Condition: if len(arr) <= 1:
    - Action: Return the list as it is.

Condition 2: Divide Step

- Find the middle index: mid = len(arr) // 2
- **Split the list into two halves:**
    - Left half → arr[:mid]
    - Right half → arr[mid:]
- Recursively call merge_sort() on both halves:
    - left = merge_sort(arr[:mid])
    - right = merge_sort(arrr[mid:])

Step 3: Merge Step

After both halves are sorted, they are merged together in ascending order.

Condition 3: Comparing elements

- Initialize three counters: i, j, and k to 0.
- While both halves have elements left:
    - Condition: while i < len(left) and j < len(right):
    - Compare elements:
        - If left[i] <= right[j] → place left[i] in the merged list.
        - Else → place right[j] in the merged list.
    - Increment counters accordingly.

Condition 4: Copy remaining elements

- If one half finishes early, copy the remaining elements from the other half.
    - Condition: while i < len(left): → copy remaining left elements.
    - Condition: while j < len(right): → copy remaining right elements.

Step 4: Return Sorted List

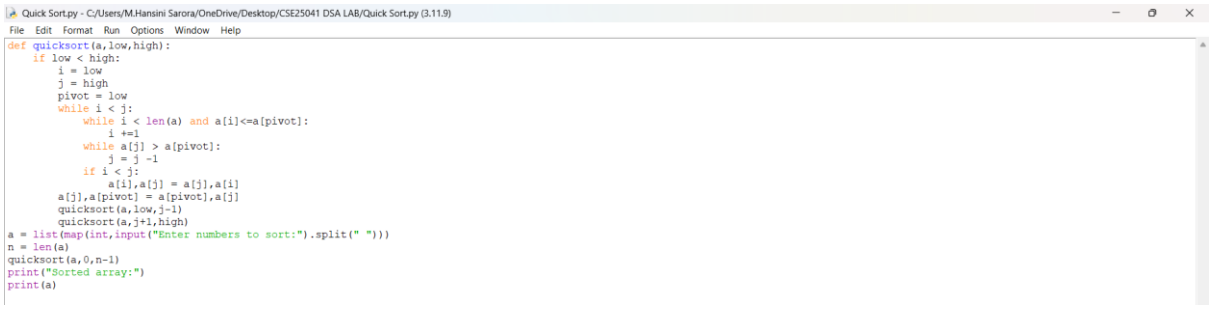
- After merging, return the sorted list to the previous recursive call.
- Finally, print the sorted list after all recursive merges are complete.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Index error List index out of range	Later, it was rectified by few corrections in while loop.
2.	Syntax error	Later, it was rectified by typing it as ':' in loop.

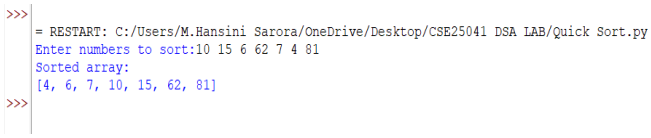
3e. AIM: Write a program to perform Merge Sort technique on the user given input.

PROGRAM:

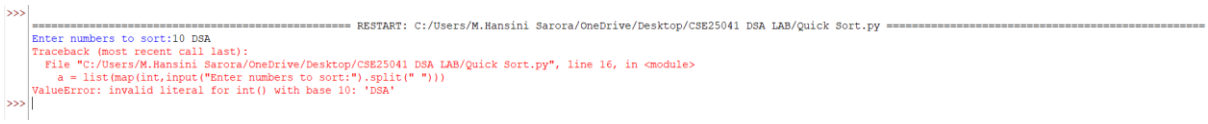


OUTPUT:

Positive Case;



Negative Case;



EXPLANATION:

Step 1: Input Phase

- The program asks the user to enter numbers separated by spaces.
- These numbers are converted into integers and stored in the list a.
- The length of the list (n) is calculated.

Step 2: Quick Sort Function

Quick Sort is a divide and conquer algorithm. It works by choosing a pivot element, partitioning the list around the pivot, and then recursively sorting the partitions.

Condition 1: Base Case

- The function runs only if low < high.

- If this condition is **false** (meaning the sublist has one or zero elements), the recursion stops because the list is already sorted.

Condition 2: Partitioning

- Variables i = low, j = high, and pivot = low are initialized.
- The pivot is chosen as the first element of the sublist.

Inner conditions:

- while i < len(a) and a[i] <= a[pivot]: i += 1
    - Condition checks if the current element is less than or equal to the pivot.
    - If true, move i forward.
    - If false, stop and prepare for a swap.
- while a[j] > a[pivot]: j -= 1
    - Condition checks if the current element is greater than the pivot.
    - If true, move j backward.
    - If false, stop and prepare for a swap.
- **Swap condition:**
    - If i < j, swap a[i] and a[j].
    - This ensures smaller elements move to the left of the pivot and larger ones to the right.

Condition 3: Pivot Placement

- After partitioning, swap the pivot element with a[j].
- This places the pivot in its correct sorted position.

Step 3: Recursive Calls

- After placing the pivot, Quick Sort is called recursively on the two partitions:

- Left sublist: quicksort (a, low, j-1)
- Right sublist: quicksort (a, j+1, high)

- Each recursive call applies the same conditions until the entire list is sorted.

Step 4: Output Phase

- Finally, the sorted list is printed.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Index Error When j becomes negative if pivot is very small compared to other elements.	Add boundary check: while j >= low and a[j] > a[pivot]: j -= 1.
2.	Unnecessary swaps When i and j point to the same element.	Add condition check before swapping: only swap if i < j.

WEEK – 4

4a. AIM: Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.

g. Delete last node. h. Count no. of nodes i. Display.

j. Exit.

PROGRAM:

Single Linked List.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Single Linked List.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next is not None:
                    temp = temp.next
                temp.next = new_node

        print("Linked List created successfully")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        new_node.next = self.head
        self.head = new_node
        print(f"Inserted {data} at the beginning.")

    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        print(f"Inserted {data} at the end.")

    # 4. Insert at Index
    def insert_at_index(self):
        data = int(input("Enter data: "))
        index = int(input("Enter index: "))

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range")
                return
            temp = temp.next

        if temp is None:
            print("Index out of range")
            return

        new_node = Node(data)

        new_node.next = temp.next
        temp.next = new_node
        print(f"Inserted {data} at index {index}.")

    # 5. Delete from Beginning
    def delete_beginning(self):

        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next
```

```
        self.head = self.head.next
        print(f"Deleted beginning node.")

    # 6. Delete from End
    def delete_end(self):

        if self.head is None:
            print("List is empty")
            return

        if self.head.next is None:
            self.head = None
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None
        print(f"Deleted last node.")

    # 7. Delete from Index
    def delete_at_index(self):

        index = int(input("Enter index: "))

        if self.head is None:
            print("List is empty")
            return

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.head = self.head.next
            return

        temp = self.head

        for i in range(index - 1):
            if temp.next is None:
                print("Index out of range")
                return
            temp = temp.next

        if temp.next is None:
            print("Index out of range")
            return

        temp.next = temp.next.next
        print(f"Deleted the node value.")

    # 8. Count Nodes
    def count_nodes(self):

        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)
    # 9. Display
    def display(self):

        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")



# Main Program

sll = SLL()

while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count Number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        sll.create()

    elif choice == 2:
        sll.insert_beginning()

    elif choice == 3:
        sll.insert_end()

    elif choice == 4:
        sll.insert_at_index()

    elif choice == 5:
        sll.delete_beginning()

    elif choice == 6:
        sll.delete_end()

    elif choice == 7:
        sll.delete_at_index()

    elif choice == 8:
        sll.count_nodes()

    elif choice == 9:
        sll.display()

    elif choice == 10:
        print("Exiting from the program...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

Positive Case;

```
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Single Linked List.py

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data: 2
Enter data: 4
Enter data: 5
Enter data: 17
Enter data: 30
Linked List created successfully

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
2 -> 4 -> 5 -> 17 -> 30 -> None
```

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 25
Inserted 25 at the beginning.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
25 -> 2 -> 4 -> 5 -> 17 -> 30 -> None
```

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 31
Inserted 31 at the end.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
25 -> 2 -> 4 -> 5 -> 17 -> 30 -> 31 -> None
```

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter data: 1
Enter index: 2
Inserted 1 at index 2.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
25 -> 2 -> 1 -> 4 -> 5 -> 17 -> 30 -> 31 -> None
```

Below is the continution of the output for a program.,

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 5
Deleted beginning node.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
2 -> 1 -> 4 -> 5 -> 17 -> 30 -> 31 -> None
```

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 6
Deleted last node.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
2 -> 1 -> 4 -> 5 -> 17 -> 30 -> None

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 7
Enter index: 1
Deleted the node value.
```

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
2 -> 4 -> 5 -> 17 -> 30 -> None

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 5

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
Exiting from the program...
```

Negative Case;

```
>>>
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Single Linked List.py ==========

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 11
Invalid choice

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 17.8
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Single Linked List.py", line 204, in <module>
    sll.insert_beginning()
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Single Linked List.py", line 31, in insert_beginning
    data = int(input("Enter data: "))
ValueError: invalid literal for int() with base 10: '17.8'
>>>
```

EXPLANATION:

◈ Class Definitions

1. Node **class**
    - Represents a single node in the linked list.
    - Each node has:
        - data → stores the value.
        - next → pointer to the next node (initially None).
2. SLL **class**
    - Manages the linked list operations.
    - Has a head pointer (initially None) that points to the first node.

◈ Operations in SLL

1. Create Linked List

- Asks user for number of nodes.
- For each node:
    - Creates a new Node.
    - If list is empty → new node becomes head.
    - Otherwise → traverses to the end and attaches the new node.

2. Insert at Beginning

- Creates a new node.
- Points its next to current head.
- Updates head to new node.

3. Insert at End

- Creates a new node.
- If list is empty → new node becomes head.
- Otherwise → traverses to last node and attaches new node.

4. Insert at Index

- Asks for index and data.

- If index = 0 → behaves like "insert at beginning".
- Otherwise → traverses to (index-1) node and inserts new node in between.

5. Delete from Beginning

- Moves head to head.next.
- Effectively removes the first node.

6. Delete from End

- If list empty → prints "List is empty".
- If only one node → sets head = None.
- Otherwise → traverses to second-last node and sets its next = None.

7. Delete from Index

- Asks for index.
- If index = 0 → removes head.
- Otherwise → traverses to (index-1) node and skips the next node.

8. Count Nodes

- Traverses through list.
- Counts nodes until None.

9. Display

- Traverses from head to end.
- Prints each node's data in data -> data -> None format.

◈ Main Program

- Runs an infinite loop with a menu:
  - Options 1–9 → perform linked list operations.
  - Option 10 → exits program.
- Uses if-elif to call respective methods.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Index out of range When inserting/deleting at an index greater than the number of nodes.	Traverse carefully; if temp becomes None before reaching index, print "Index out of range" and stop.
2.	Logical error in insertion if index is negative.	Validate index: if index < 0: print ("Invalid index").
3.	Wrong output format In display (missing None at end).	After traversal, always print "None" to mark end of list.
4.	Indentation Error In the Count node part	It was rectified later, By maintaining proper indentation.

4b. AIM: Write a python program to implement a double linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.

g. Delete last node. h. Count no. of nodes i. Display.

j. Exit.

PROGRAM:

Double Linked List.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Double Linked List.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoubleLinkedList:

    def __init__(self):
        self.head = None
```

```
# a. Create Linked List
def create(self):
    n = int(input("Enter number of nodes: "))

    for i in range(n):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

    print("Double Linked List created successfully.")

# b. Insert at Beginning
def insert_beginning(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    new_node.next = self.head

    if self.head is not None:
        self.head.prev = new_node

    self.head = new_node

    print(f"Inserted {data} at the beginning.")

# c. Insert at End
def insert_end(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    print("Double Linked List created successfully.")

# b. Insert at Beginning
def insert_beginning(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    new_node.next = self.head

    if self.head is not None:
        self.head.prev = new_node

    self.head = new_node

    print(f"Inserted {data} at the beginning.")

# c. Insert at End
def insert_end(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    print(f"Inserted {data} at the end.")

# d. Insert at Specific Index
def insert_at_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))

    if index < 0:
        print("Invalid index.")
        return

    new_node = Node(data)

    if index == 0:
        new_node.next = self.head

        if self.head is not None:
            self.head.prev = new_node

        self.head = new_node

        print(f"Inserted {data} at index {index}.")
        return

    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Index out of range.")
            return

        temp = temp.next

    if temp is None:
        print("Index out of range.")
        return

    new_node.next = temp.next
    new_node.prev = temp

    if temp.next is not None:
        temp.next.prev = new_node

    temp.next = new_node

    print(f"Inserted {data} at index {index}.")
```

```
    # e. Delete by Value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("Doubly Linked List is empty.")
            return

        temp = self.head

        while temp is not None:
            if temp.data == value:
                break
            temp = temp.next

        if temp is None:
            print(f"{value} not found in the Linked List.")
            return

        if temp.prev is None:
            self.head = temp.next

            if self.head is not None:
                self.head.prev = None

        else:
            temp.prev.next = temp.next

            if temp.next is not None:
                temp.next.prev = temp.prev

        print(f"Deleted {value}.")

    # f. Delete First Node
    def delete_first(self):
        if self.head is None:
            print("Doubly Linked List is empty.")
            return

        deleted = self.head.data

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print(f"Deleted first node: {deleted}")

    # g. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("Doubly Linked List is empty.")
            return

        temp = self.head

        if temp.next is None:
            deleted = temp.data
            self.head = None
            print(f"Deleted last node: {deleted}")
            return

        while temp.next is not None:
            temp = temp.next

        deleted = temp.data

        temp.prev.next = None

        print(f"Deleted last node: {deleted}")

    # h. Count Number of Nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

     # i. Display
     def display(self):
         if self.head is None:
             print("Doubly Linked List is empty.")
             return

         temp = self.head

         print("Doubly Linked List:", end=" ")

         while temp is not None:
             print(temp.data, end=" <-> ")
             temp = temp.next

         print("None")


# Main Program
dll = DoubleLinkedList()

while True:
```

```
print("\n----- DOUBLE LINKED LIST -----")
print("1. Create Linked List")
print("2. Insert at Beginning")
print("3. Insert at End")
print("4. Insert at Specific Index")
print("5. Delete by Value")
print("6. Delete First Node")
print("7. Delete Last Node")
print("8. Count Number of Nodes")
print("9. Display")
print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    dll.create()

elif choice == 2:
    dll.insert_beginning()

elif choice == 3:
    dll.insert_end()

elif choice == 4:
    dll.insert_at_index()

elif choice == 5:
    dll.delete_by_value()

elif choice == 6:
    dll.delete_first()

elif choice == 7:
    dll.delete_last()

elif choice == 8:
    dll.count_nodes()

elif choice == 9:
    dll.display()

elif choice == 10:
    print("Exiting Program...")
    break

else:
    print("Invalid choice. Please try again.")
```

OUTPUT:

Positive Case;

```
>>>
= RESTART: C:\Users\M.Hansini Sarora\OneDrive\Desktop\CSE25041 DSA LAB\Double Linked List.py

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 6
Enter data: 12
Enter data: 4
Enter data: 75
Enter data: 86
Enter data: 18
Enter data: 15
Double Linked List created successfullv.
```

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List: 12 <-> 4 <-> 75 <-> 86 <-> 18 <-> 15 <-> None
```

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 85
Inserted 85 at the beginning.

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List: 85 <-> 12 <-> 4 <-> 75 <-> 86 <-> 18 <-> 15 <-> None
```

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 95
Inserted 95 at the end.

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List: 85 <-> 12 <-> 4 <-> 75 <-> 86 <-> 18 <-> 15 <-> 95 <-> None
```

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 4
Enter index: 5
Enter data: 64
Inserted 64 at index 5.
```

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 5
Enter value to delete: 75
Deleted 75.

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List: 85 <-> 12 <-> 4 <-> 86 <-> 64 <-> 18 <-> 15 <-> 95 <-> None

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 6
Deleted first node: 85
```

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List: 12 <-> 4 <-> 86 <-> 64 <-> 18 <-> 15 <-> 95 <-> None

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 7
Deleted last node: 95

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List: 12 <-> 4 <-> 86 <-> 64 <-> 18 <-> 15 <-> None
```

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 6
```

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List: 12 <-> 4 <-> 86 <-> 64 <-> 18 <-> 15 <-> None

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 10
Exiting Program...
```

Negative Case;

```
>>>
========== RESTART: C:\Users\M.Hansini Sarora\OneDrive\Desktop\CSE25041 DSA LAB\Double Linked List.py ==========

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 14
Invalid choice. Please try again.

----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 4
Enter index: 25
Enter data: 14
Index out of range.
```

EXPLANATION:

1. Node Class

- Each node stores:
  - data: the actual value.
  - prev: pointer to the previous node.
  - next: pointer to the next node.

- This structure allows traversal in both directions (forward and backward).

2. Double Linked List Class

This class manages the linked list. It starts with self.head = None (empty list). It provides several operations:

a. Create Linked List

- User specifies how many nodes to create.
- Each new node is linked at the end.
- prev pointer connects the new node to the previous one.

b. Insert at Beginning

- A new node is created.
- Its next points to the current head.
- The old head's prev points back to the new node.
- The new node becomes the head.

c. Insert at End

- Traverse to the last node.
- Attach the new node at the end.
- Update prev of the new node to point to the last node.

d. Insert at Specific Index

- Traverse until the given index.
- Insert the new node between two nodes.
- Adjust both next and prev pointers to maintain the chain.

e. Delete by Value

- Search for the node with the given value.
- If found:

- If it’s the head, move head to the next node.
- Otherwise, adjust prev.next and next.prev to bypass the node.

- This removes the node from the chain.

f. Delete First Node

- Move head to the second node.
- Set its prev to None.

g. Delete Last Node

- Traverse to the last node.
- Remove it by setting its previous node’s next to None.

h. Count Nodes

- Traverse the list, incrementing a counter.
- Prints the total number of nodes.

i. Display

- Traverse from head to end.
- Print each node’s data in the format: data <-> data <-> ... None

3. Main Program (Menu-Driven)

- Runs an infinite loop with options:
    1. Create Linked List
    2. Insert at Beginning
    3. Insert at End
    4. Insert at Specific Index
    5. Delete by Value
    6. Delete First Node
    7. Delete Last Node

8. Count Nodes
9. Display
10. Exit

- Based on user choice, the corresponding method is executed.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Invalid index entered (negative or too large).	Add a check: if index < 0 → print "Invalid index"; if traversal reaches None before index → print "Index out of range".
2.	Deleting from an empty list.	Before deletion, check if self.head is None: → print "Doubly Linked List is empty."
3.	Value not found during deletion by value.	Traverse list; if node not found → print "value not found in the Linked List."
4.	Insertion at index when list is empty but index > 0.	Handle separately: if self.head is None and index! = 0 → print "Index out of range."

4c. AIM: Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.

g. Delete last node. h. Count no. of nodes i. Display.

j. Display head and tail. k. Print data from tail to head. l. Exit.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:

    def __init__(self):
        self.head = None

    # a. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                temp.next = new_node
                new_node.next = self.head

        print("Circular Linked List created successfully.")

    # b. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next
          new_node.next = self.head
          temp.next = new_node
          self.head = new_node

       print(f"Inserted {data} at the beginning.")

    # c. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new_node
            new_node.next = self.head

        print(f"Inserted {data} at the end.")

    # d. Insert at Specific Index
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        if index < 0:
            print("Invalid index.")
            return

        if index == 0:
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next
```

```
            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

        print(f"Inserted {data} at index {index}.")
        return

    if self.head is None:
        print("Index out of range.")
        return

    new_node = Node(data)
    temp = self.head

    for i in range(index - 1):
        temp = temp.next

        if temp == self.head:
            print("Index out of range.")
            return

    new_node.next = temp.next
    temp.next = new_node

    print(f"Inserted {data} at index {index}.")

# e. Delete by Value
def delete_by_value(self):
    value = int(input("Enter value to delete: "))

    if self.head is None:
        print("Circular Linked List is empty.")
        return

    if self.head.data == value:

        if self.head.next == self.head:
            self.head = None

        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

                temp.next = self.head.next
                self.head = self.head.next

            print(f"Deleted {value}.")
            return

    temp = self.head

    while temp.next != self.head:

        if temp.next.data == value:
            temp.next = temp.next.next
            print(f"Deleted {value}.")
            return

        temp = temp.next

    print(f"{value} not found in the Linked List.")

# f. Delete First Node
def delete_first(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return

    deleted = self.head.data

    if self.head.next == self.head:
        self.head = None

    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = self.head.next
        self.head = self.head.next

    print(f"Deleted first node: {deleted}")
```

```
    # g. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return

        if self.head.next == self.head:
            deleted = self.head.data
            self.head = None
            print(f"Deleted last node: {deleted}")
            return

        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next

        deleted = temp.next.data
        temp.next = self.head

        print(f"Deleted last node: {deleted}")

    # h. Count Number of Nodes
    def count_nodes(self):
        if self.head is None:
            print("Number of nodes: 0")
            return

        count = 0
        temp = self.head

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)

# i. Display
def display(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return

    temp = self.head

    print("Circular Linked List:", end=" ")

    while True:
        print(temp.data, end=" -> ")
        temp = temp.next

        if temp == self.head:
            break

    print("HEAD")

# j. Display Head and Tail
def display_head_tail(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    tail = temp

    print("Head:", self.head.data)
    print("Tail:", tail.data)
```

```
    # k. Print Data from Tail to Head
    def display_tail_to_head(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return

        tail = self.head

        while tail.next != self.head:
            tail = tail.next

        print("Data from Tail to Head:", end=" ")

        current = tail

        while True:
            print(current.data, end=" ")

            if current == self.head:
                break

            temp = self.head

            while temp.next != current:
                temp = temp.next

            current = temp

        print()


cll = CircularLinkedList()

while True:

    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specific Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display")
    print("10. Display Head and Tail")
    print("11. Print Data from Tail to Head")
    print("12. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()

    elif choice == 2:
        cll.insert_beginning()

    elif choice == 3:
        cll.insert_end()

    elif choice == 4:
        cll.insert_at_index()

    elif choice == 5:
        cll.delete_by_value()

    elif choice == 6:
        cll.delete_first()

    elif choice == 7:
        cll.delete_last()

    elif choice == 8:
        cll.count_nodes()

    elif choice == 9:
        cll.display()

    elif choice == 10:
        cll.display_head_tail()
```

```
    elif choice == 11:
        cll.display_tail_to_head()

    elif choice == 12:
        print("Exiting program....")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

Positive Case;

```
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Circular Linked List.py ==========

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data: 14
Enter data: 25
Enter data: 36
Enter data: 45
Enter data: 58
Circular Linked List created successfully.

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 14 -> 25 -> 36 -> 45 -> 58 -> HEAD
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 2
Enter data: 64
Inserted 64 at the beginning.

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 64 -> 14 -> 25 -> 36 -> 45 -> 58 -> HEAD
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 3
Enter data: 72
Inserted 72 at the end.

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 64 -> 14 -> 25 -> 36 -> 45 -> 58 -> 72 -> HEAD
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 4
Enter index: 5
Enter data: 83
Inserted 83 at index 5.

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 64 -> 14 -> 25 -> 36 -> 45 -> 83 -> 58 -> 72 -> HEAD
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 5
Enter value to delete: 58
Deleted 58.
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 64 -> 14 -> 25 -> 36 -> 45 -> 83 -> 72 -> HEAD

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 6
Deleted first node: 64
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 14 -> 25 -> 36 -> 45 -> 83 -> 72 -> HEAD

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 7
Deleted last node: 72
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 14 -> 25 -> 36 -> 45 -> 83 -> HEAD

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 10
Head: 14
Tail: 83
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 15 -> 14 -> 25 -> 36 -> 37 -> 45 -> 58 -> 48 -> HEAD
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 11
Data from Tail to Head: 83 45 36 25 14

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 12
Exiting program....
```

Negative Case;

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 5
Enter value to delete: 10.58
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Circular Linked List.py", line 319, in <module>
    cll.delete_by_value()
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Circular Linked List.py", line 122, in delete_by_value
    value = int(input("Enter value to delete: "))
ValueError: invalid literal for int() with base 10: '10.58'
```

EXPLANATION:

1. Node Class

- Each node stores:
    - data: the value.
    - next: pointer to the next node.
- In a circular linked list, the last node's next points back to the head, forming a circle.

2. Circular Linked List Class

This class manages the linked list. It starts with self.head = None (empty list). It provides several operations:

a. Create Linked List

- User specifies number of nodes.
- Each new node is linked at the end.
- The last node's next points back to the head, ensuring circularity.

b. Insert at Beginning

- A new node is created.
- It becomes the new head.
- The last node's next is updated to point to this new head.

c. Insert at End

- Traverse to the last node.
- Attach the new node after it.
- New node's next points back to head.

d. Insert at Specific Index

- If index = 0 → same as inserting at beginning.
- Otherwise, traverse to the given position.
- Insert the new node between two nodes.
- Maintain circularity by adjusting next pointers.

e. Delete by Value

- Search for the node with the given value.
- If it's the head:
    - If only one node exists → set head to None.
      Otherwise, adjust last node's next to point to new head.
- If it's in the middle → bypass the node by linking previous node's next to the next node.

f. Delete First Node

- If only one node → set head to None.
- Otherwise, move head to the second node and update last node's next.

g. Delete Last Node

- Traverse to the second-last node.
- Make its next point to head.
- Removes the last node.

h. Count Nodes

- Traverse until you return to head.
- Count nodes during traversal.

i. Display

- Print nodes starting from head.
- Stop when traversal returns to head.
- Shows circularity by ending with "HEAD".

j. Display Head and Tail

- Head = first node.
- Tail = last node (whose next points to head).
- Prints both values.

k. Print Data from Tail to Head

- Find tail.
- Traverse backward by repeatedly searching for the node whose next points to the current node.
- Prints values in reverse order until reaching head.

3. Main Program (Menu-Driven)

- Runs an infinite loop with give options.
- Based on user choice, the corresponding method is executed.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Display on empty list.	Add condition: if self.head is None: → print "Circular Linked List is empty."
2.	Deleting from an empty list.	Before deletion, check if self.head is None: → print "Circular Linked List is empty."
3.	Infinite loop in traversal (if circularity broken).	Ensure every insertion/deletion maintains circularity: last node's next must always point to self.head.
4.	Reverse traversal (Tail to Head) inefficient.	Current implementation repeatedly searches for predecessor. Rectify by using a doubly circular linked list if reverse traversal is frequent.

WEEK – 5

5. AIM: Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

PROGRAM:

```
class StackArray:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            for i in range(self.top, -1, -1):
                print(self.stack[i])


size = int(input("Enter size of stack: "))
s = StackArray(size)

while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")
```

OUTPUT:

Positive Case;

```
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Stack using Arrays.py
Enter size of stack: 5

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 25
25 pushed into the stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 30
30 pushed into the stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 17
17 pushed into the stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 4
4 pushed into the stack
```

```
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 5
5 pushed into the stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
5
4
17
30
25

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
5 popped from the stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
4
17
30
25
```

```
--

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 4

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
4
17
30
25

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program...
```

Negative Case;

```
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 4
Stack Overflow
```

```
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Stack Underflow
```

Stack Using Linked List;

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class StackLinkedList:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
```

```
        else:
            popped = self.top.data
            self.top = self.top.next
            print(popped, "popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next

sll = StackLinkedList()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        sll.push(item)
    elif choice == 2:
        sll.pop()
    elif choice == 3:
        sll.peek()
    elif choice == 4:
        sll.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")
```

OUTPUT:

Positive Case;

```
>>>
    = RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Stack using LinkedList.py

    --- STACK USING LINKED LIST ---
    1. Push
    2. Pop
    3. Peek
    4. Display
    5. Exit
    Enter your choice: 1
    Enter element to push: 4
    4 pushed into the stack

    --- STACK USING LINKED LIST ---
    1. Push
    2. Pop
    3. Peek
    4. Display
    5. Exit
    Enter your choice: 1
    Enter element to push: 5
    5 pushed into the stack

    --- STACK USING LINKED LIST ---
    1. Push
    2. Pop
    3. Peek
    4. Display
    5. Exit
    Enter your choice: 1
    Enter element to push: 17
    17 pushed into the stack

    --- STACK USING LINKED LIST ---
    1. Push
    2. Pop
    3. Peek
    4. Display
    5. Exit
    Enter your choice: 1
    Enter element to push: 25
    25 pushed into the stack
```

```
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
2 popped from the stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
30
25
17
5
4

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 30

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program...
```

```
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 30
30 pushed into the stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 2
2 pushed into the stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
2
30
25
17
5
4
```

Negative Case;

```
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 10.2
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Stack using LinkedList.py", line 54, in <module>
    item = int(input("Enter element to push: "))
ValueError: invalid literal for int() with base 10: '10.2'
>>
```

EXPLANATION:

□ Stack Using Array

Structure:

- Uses a Python list (self.stack) to store elements.
- self.top keeps track of the index of the last pushed element.
- self.size defines the maximum capacity.

Operations:

1. Push
    - Check if top == size - 1 → Overflow.
    - Otherwise, increment top and insert the element at that position.
2. Pop
    - Check if top == -1 → Underflow (empty stack).
    - Otherwise, remove the element at top, decrement top.
3. Peek
    - Shows the element at top without removing it.
4. Display
    - Prints elements from top down to 0 (top to bottom order).

□ Stack Using Linked List

Structure:

- Each element is a Node with:
    - data (value)
    - next (pointer to the next node).
- self.top points to the most recently pushed node.

Operations:

1. Push
    - Create a new node.
    - Link its next to the current top.
    - Update top to this new node.
2. Pop
    - If top is None → Underflow.
    - Otherwise, remove the node at top and update top to top.next.
3. Peek
    - Shows the data at top.
4. Display
    - Traverse from top downwards, printing each node's data.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	def _init_(self, size):	Python expects **two underscores** before and after init.
2.	Display on empty stack.	Add condition: if self.top is None: → print "Stack is empty".
3.	Stack Overflow when pushing beyond the defined size	Add a check: if self.top == self.size - 1: → print "Stack Overflow".
4.	Stack Underflow when popping from empty stack	Add a check: if self.top is None: → print "Stack Underflow".
5.	Syntax error	Later it was rectified in the for loop of display section.

WEEK – 6

6. AIM: To implement Queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu driven approach.

6a. Using Array, Linked list, Data structures.

6b. Circular Queue.

PROGRAM:

Queue using Array.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Queue using Array.py (3.13.7)

File Edit Format Run Options Window Help

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1

            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])

size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

Positive Case;

```
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Queue using Array.py
Enter the size of Queue: 4

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 45
45 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 52
52 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
45
52

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
45 deleted from the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
52

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 52

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting Program
>>>
```

Negative Case;

```
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Queue using Array.py
Enter the size of Queue: 1.0
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Queue using Array.py", line 48, in <module>
    size = int(input("Enter the size of Queue: "))
ValueError: invalid literal for int() with base 10: '1.0'
>>>
```

Linked List.,

Queue using Linked List.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Queue using Linked List.py (3.13.7)

File Edit Format Run Options Window Help

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None

            print(f"{x} deleted from the queue")
            self.front += 1

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])
```

```
size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
        print("\n----- Queue Menu -----")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            item = int(input("Enter the element to enqueue: "))
            q.enqueue(item)

        elif choice == 2:
            q.dequeue()

        elif choice == 3:
            q.peek()

        elif choice == 4:
            q.display()

        elif choice == 5:
            print("Exiting Program")
            break

        else:
            print("Invalid Choice!")
```

OUTPUT:

Positive Case;

```
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Queue using Linked List.py ==========
Enter the size of Queue: 5

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 14
14 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 41
41 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 09
9 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
14
41
9

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
14 deleted from the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
41
9

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 41

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting Program
>>>
```

Negative Case;

```
>>>
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Queue using Linked List.py ==========
Enter the size of Queue: 2

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 4
4 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 5
5 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 6
Queue Overflow
```

EXPLANATION:

🔷 What is a Queue?

- A Queue is a linear data structure that follows the FIFO (First In First Out) principle.
- Elements are inserted at the rear and removed from the front.

Arrays:

🔷 **Components in the Code**

1. **Initialization (__init__)**
    - Creates a queue of fixed size.
    - Uses an array (self.queue) to store elements.
    - front and rear pointers track the first and last elements.
    - Both start at -1 (empty queue).
2. **Enqueue (Insertion)**
    - Adds an element at the rear.
    - If rear == size - 1, → **Overflow** (queue full).
    - If queue is empty (front == -1), set front = 0.
    - Increment rear and insert the element.
3. **Dequeue (Deletion)**

- Removes an element from the front.
- If queue is empty (front == -1 or front > rear), → **Underflow**.
- Deletes the element at front, increments front.
- If all elements are deleted (front > rear), reset both pointers to -1.

4. **Peek**

- Shows the element at the front without removing it.

- If empty, prints "Queue is empty".

5. **Display**

- Prints all elements between front and rear.
- If empty, prints "Queue is empty".

◈ Menu-Driven Program

- User chooses operations from a menu:
    1. **Enqueue** → Insert element.
    2. **Dequeue** → Remove element.
    3. **Peek** → Show front element.
    4. **Display** → Show all elements.
    5. **Exit** → End program.
- The loop continues until the user selects **Exit**.

◈ Characteristics of This Queue

- It is a **Linear Queue** (not circular).
- **Overflow** occurs when rear == size - 1.
- **Underflow** occurs when queue is empty.
- Once rear reaches the end, no more insertions are possible even if earlier slots are freed (limitation of linear queue).

Linked List:

◈ Structure in the Code

1. **Initialization (__init__)**
    - Creates a queue of fixed size (default = 5).
    - Uses an array (self.queue) to store elements.
    - front and rear pointers track the first and last elements.
    - Both start at -1 (empty queue).

2. **Enqueue (Insertion)**
    - Adds an element at the rear.
    - If rear == size - 1, → **Overflow** (queue full).
    - If queue is empty (front == -1), set front = 0.
    - Increment rear and insert the element.
3. **Dequeue (Deletion)**
    - Removes an element from the front.
    - If queue is empty (front == -1), → **Underflow**.
    - Deletes the element at front, increments front.
    - If all elements are deleted (front > rear), reset both pointers to -1.
4. **Peek**
    - Shows the element at the front without removing it.
    - If empty, prints "Queue is empty".
5. **Display**
    - Prints all elements between front and rear.
    - If empty, prints "Queue is empty".

◈ Characteristics of This Queue

- It is a **Linear Queue** (not circular).
- **Overflow** occurs when rear == size - 1.
- **Underflow** occurs when queue is empty (front == -1).
- Once rear reaches the end, no more insertions are possible even if earlier slots are freed (limitation of linear queue).

6b. To implement Circular Queue and its operation**.**

PROGRAM:

Circular Queue.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Circular Queue.py (3.13.7)

File Edit Format Run Options Window Help

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size
```

```
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)

while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.display()
    elif choice == 4:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

Positive Case;

```
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Circular Queue.py ==========
Enter the size of the queue: 5

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 2
2 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 17
17 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 30
30 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
The elements of the queue are:
2
17
30

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
2 deleted from the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
The elements of the queue are:
17
30

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Program terminated.
>>>
```

Negative Case;

```
>>>
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Circular Queue.py ==========
Enter the size of the queue: 1

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 5
Invalid choice
```

EXPLANATION:

◆ Concept of Circular Queue

- A **Circular Queue** is a linear data structure that follows the **FIFO (First In First Out)** principle.
- Unlike a simple linear queue, the last position connects back to the first position, forming a circle.

- This design solves the problem of wasted space in a linear queue when elements are deleted from the front.

◈ Structure in the Code

1. **Initialization (__init__)**
    - Creates a queue of fixed size.
    - Uses an array (self.queue) to store elements.
    - front and rear pointers track the first and last elements.
    - Both start at -1 (empty queue).
2. **Enqueue (Insertion)**
    - Adds an element at the rear.
    - Overflow occurs if (rear + 1) % size == front (queue full).
    - If queue is empty (front == -1), both front and rear are set to 0.
    - Otherwise, rear moves forward using modulo arithmetic: (rear + 1) % size.
3. **Dequeue (Deletion)**
    - Removes an element from the front.
    - Underflow occurs if front == -1 (queue empty).
    - If only one element is left (front == rear), both pointers reset to -1.
    - Otherwise, front moves forward using (front + 1) % size.
4. **Display**
    - Prints all elements from front to rear using modulo arithmetic.
    - Traverses circularly until it reaches the rear.

◈ Menu-Driven Program

- User chooses operations from a menu:

1. **Enqueue** → Insert element.
2. **Dequeue** → Remove element.
3. **Display** → Show all elements.
4. **Exit** → End program.

- The loop continues until the user selects **Exit**.

◈ Characteristics of Circular Queue

- **Efficient use of memory**: Freed spaces are reused.
- **FIFO principle** maintained.
- **Overflow condition:** (rear + 1) % size == front.
- **Underflow condition:** front == -1.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Redundant underflow check → Condition if self.front == -1 or self.front > self.rear: is slightly redundant because resetting pointers already handles empty state.	Simplify to if self.front == -1.
2.	Missing underflow condition → In dequeue, only checks if self.front == -1. If multiple deletions occur, front > rear is not handled, leading to incorrect behavior.	Add condition: if self.front == -1 or self.front > self.rear.
3.	Circular Queue: Display traversal limitation → Display prints values but not indices, making debugging harder.	Optional improvement: print both index and value.

WEEK – 7

7. AIM: Write a python program to implement binary tree

7a. Using Array, and

7b. Linked List

With all of its operation.

7a. Using Array,

PROGRAM:

Binary tree using Array.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Array.py (3.13.7)

File Edit Format Run Options Window Help

```
class BinaryTree:
    def __init__(self, size=100):
        self.tree = [None] * size
        self.size = size

    def insert_root(self, value):
        if self.tree[0] is not None:
            print("Root already exists!")
        else:
            self.tree[0] = value
            print("Root inserted successfully.")

    def insert_left(self, parent, value):
        if parent not in self.tree:
            print("Parent not found!")
            return

        i = self.tree.index(parent)
        left = 2 * i + 1

        if left >= self.size:
            print("Array capacity exceeded!")
        elif self.tree[left] is not None:
            print("Left child already exists!")
        else:
            self.tree[left] = value
            print("Left child inserted successfully.")

    def insert_right(self, parent, value):
        if parent not in self.tree:
            print("Parent not found!")
            return

        i = self.tree.index(parent)
        right = 2 * i + 2

        if right >= self.size:
            print("Array capacity exceeded!")
        elif self.tree[right] is not None:
            print("Right child already exists!")
        else:
            self.tree[right] = value
            print("Right child inserted successfully.")

    def search(self, value):
```

Binary tree using Array.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Array.py (3.13.7)

File Edit Format Run Options Window Help

```
    def display(self):
        if all(x is None for x in self.tree):
            print("Tree is empty.")
            return

        print("Array representation:")
        for i, value in enumerate(self.tree):
            if value is not None:
                print(f"Index {i}: {value}")

    def preorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return
        print(self.tree[i], end=" ")
        self.preorder(2 * i + 1)
        self.preorder(2 * i + 2)

    def inorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return
        self.inorder(2 * i + 1)
        print(self.tree[i], end=" ")
        self.inorder(2 * i + 2)

    def postorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return
        self.postorder(2 * i + 1)
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")

    def delete(self, value):
        if value not in self.tree:
            print("Element not found!")
            return

        i = self.tree.index(value)

        # A node can be deleted only if it has no children
        left = 2 * i + 1
        right = 2 * i + 2

        if ((left < self.size and self.tree[left] is not None)
                or (right < self.size and self.tree[right] is not None)):
            print("Delete its children first!")
        else:
```

Binary tree using Array.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Array.py (3.13.7)

File Edit Format Run Options Window Help

```
            print("Delete its children first!")
        else:
            self.tree[i] = None
            print("Element deleted successfully.")


bt = BinaryTree()

while True:
    print("\n--- BINARY TREE MENU ---")
    print("1. Insert Root")
    print("2. Insert Left Child")
    print("3. Insert Right Child")
    print("4. Search Element")
    print("5. Display Tree")
    print("6. Preorder Traversal")
    print("7. Inorder Traversal")
    print("8. Postorder Traversal")
    print("9. Delete Element")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter root value: "))
        bt.insert_root(value)

    elif choice == 2:
        parent = int(input("Enter parent value: "))
        value = int(input("Enter left child value: "))
        bt.insert_left(parent, value)

    elif choice == 3:
        parent = int(input("Enter parent value: "))
        value = int(input("Enter right child value: "))
        bt.insert_right(parent, value)

    elif choice == 4:
        value = int(input("Enter element to search: "))
        bt.search(value)

    elif choice == 5:
        bt.display()

    elif choice == 6:
        print("Preorder:", end=" ")
        bt.preorder()
```

```
    elif choice == 6:
        print("Preorder:", end=" ")
        bt.preorder()
        print()

    elif choice == 7:
        print("Inorder:", end=" ")
        bt.inorder()
        print()

    elif choice == 8:
        print("Postorder:", end=" ")
        bt.postorder()
        print()

    elif choice == 9:
        value = int(input("Enter element to delete: "))
        bt.delete(value)

    elif choice == 10:
        print("Exiting program.")
        break

    else:
        print("Invalid choice! Try again.")
```

OUTPUT:

Positive Case;

IDLE Shell 3.13.7

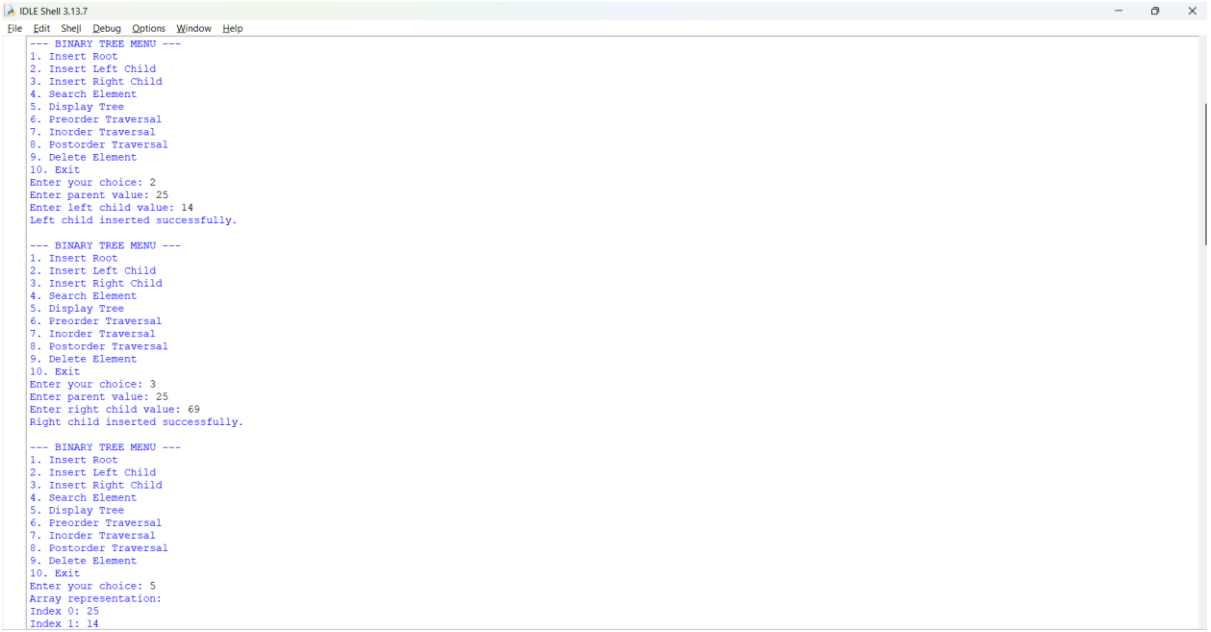
File Edit Shell Debug Options Window Help

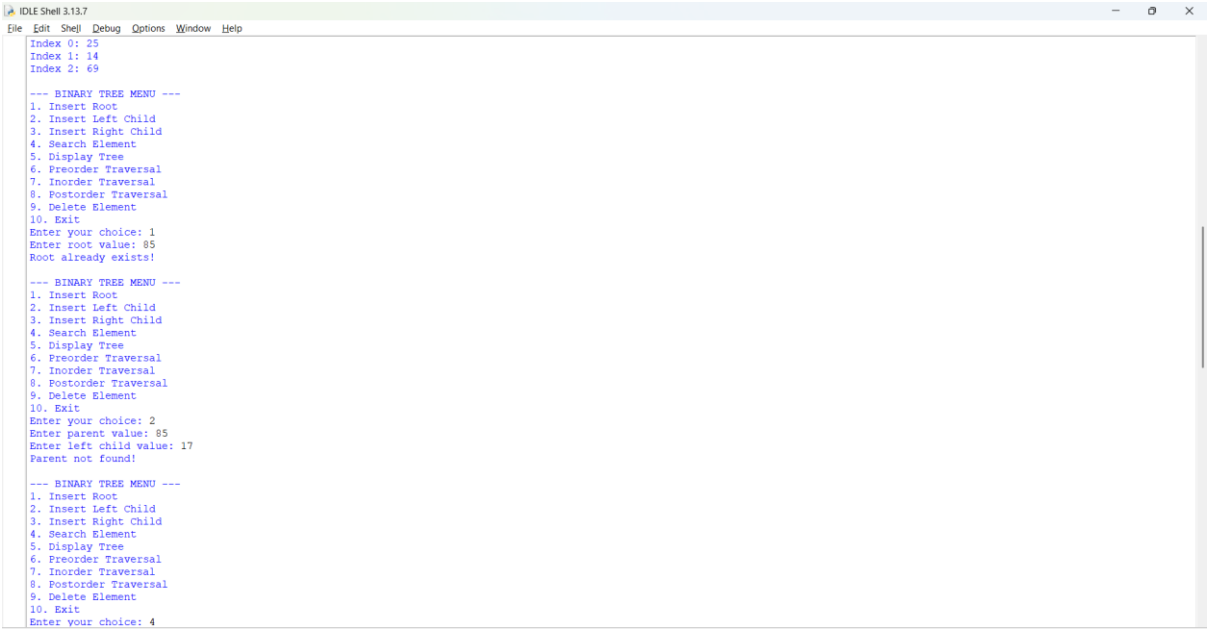
```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Array.py

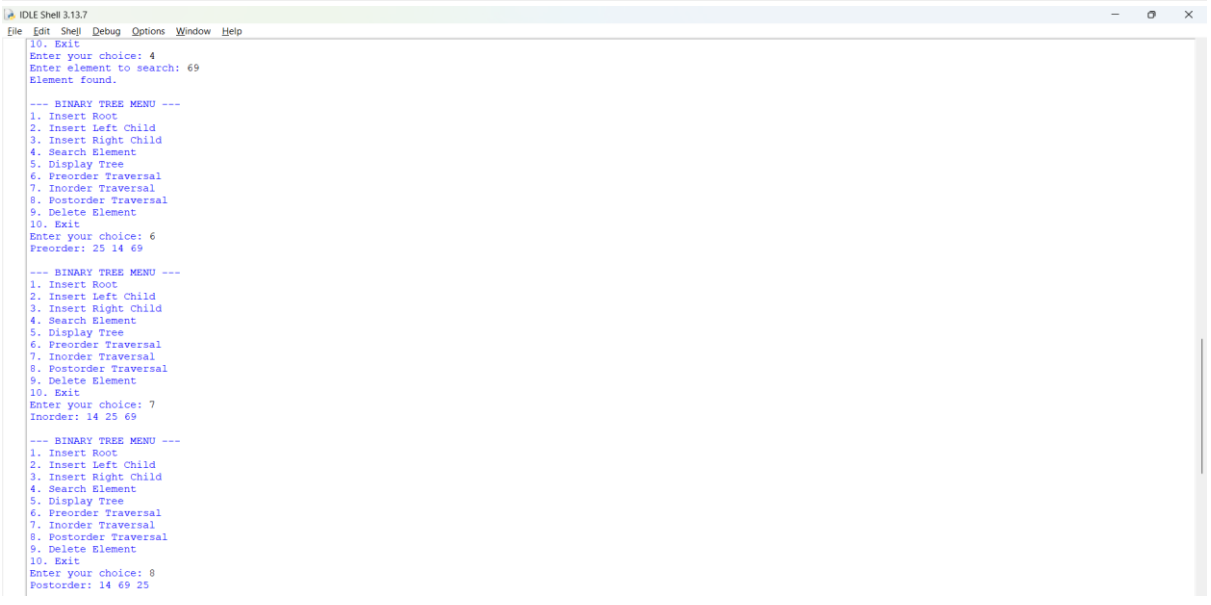
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 1
Enter root value: 25
Root inserted successfully.

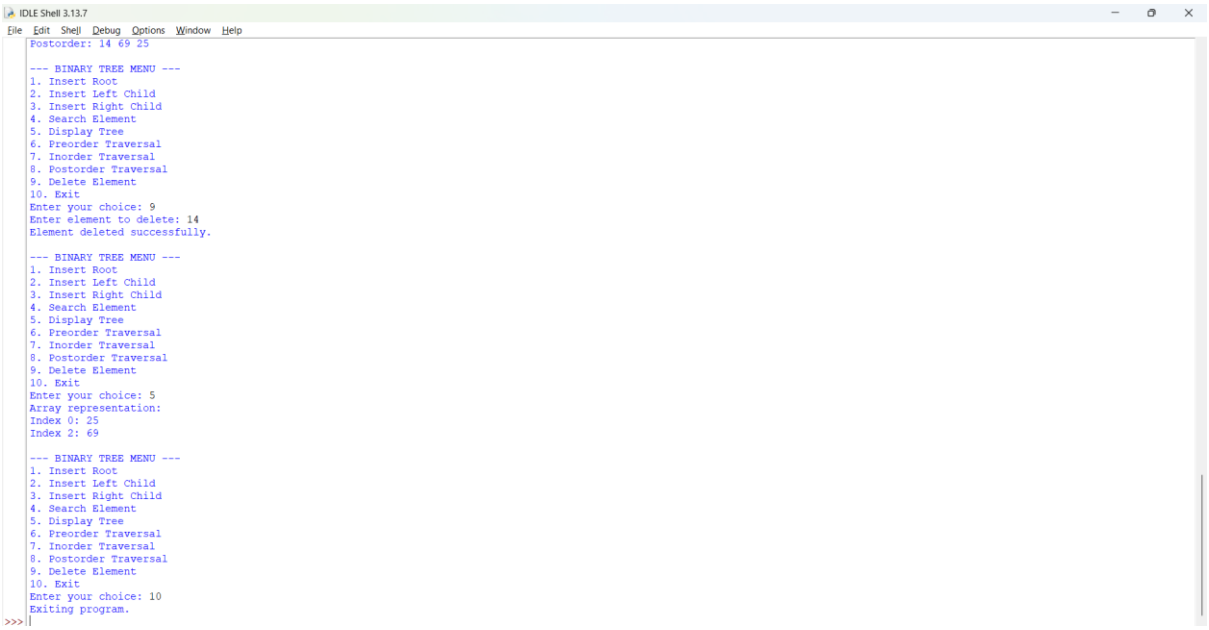
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 25
Enter left child value: 14
Left child inserted successfully.

--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
```

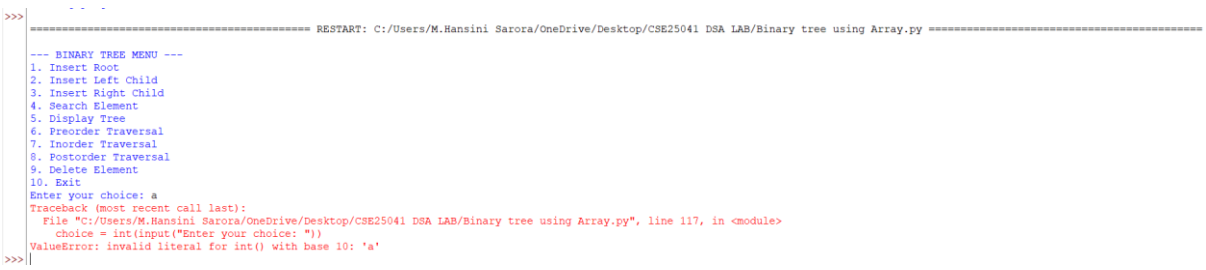

IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 25
Enter left child value: 14
Left child inserted successfully.
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 3
Enter parent value: 25
Enter right child value: 69
Right child inserted successfully.
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 5
Array representation:
Index 0: 25
Index 1: 14


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Index 0: 25
Index 1: 14
Index 2: 69
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 1
Enter root value: 85
Root already exists!
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 85
Enter left child value: 17
Parent not found!
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 4


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
10. Exit
Enter your choice: 4
Enter element to search: 69
Element found.
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 6
Preorder: 25 14 69
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 7
Inorder: 14 25 69
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 8
Postorder: 14 69 25



Negative Case;



EXPLANATION:

1. Theoretical Foundation: Array-Based Binary Trees

Unlike pointer-based implementations (using nodes and references), an array-based binary tree maps nodes into a linear list (array) based on their hierarchical positions.

Assuming 0-based indexing where the root is at index 0, the relationships for any node at index i are defined mathematically:

- Left Child Index: $2 \times i + 1$
- Right Child Index: $2 \times i + 2$
- Parent Index: $(i - 1)//2$ (integer division)

Advantages:

- No extra memory overhead for storing pointers/references.
- Direct access to children and parents using simple arithmetic.

Disadvantages:

- Fixed Size: The array capacity must be predefined (size=100), risking Array capacity exceeded errors if exceeded.
- Space Inefficiency: If the tree is skewed or sparse (e.g., a left-skewed tree), a significant portion of array elements remain None, wasting memory.

2. Breakdown of Class Methods

Initialization (__init__)

Sets up a fixed-size Python list initialized with None values to represent empty tree slots.

Insertion (insert_root, insert_left, insert_right)

- insert_root: Places the value at index 0 if it is currently vacant.
- insert_left & insert_right: First searches for the parent node's index using Python's. index() method. It then calculates the child's index ($2i + 1$ or $2i + 2$). Before insertion, it validates three conditions:
    1. Does the parent exist?
    2. Does the calculated child index stay within the array bounds (size)?
    3. Is the target child position currently empty (None)?

Search & Display (search, display)

- search: Checks if the target value exists anywhere in the array using the in operator.
- display: Iterates through the list, ignoring None entries, and prints the indices and values of all active nodes.

Tree Traversals (preorder, inorder, postorder)

These are Depth-First Search (DFS) algorithms implemented recursively using array indices:

- Preorder (Root, Left, Right): Processes the current index i, then recursively visits the left child $(2i + 1)$ and right child $(2i + 2)$.
- Inorder (Left, Root, Right): Recursively visits the left child, processes the current index i, then visits the right child.
- Postorder (Left, Right, Root): Recursively visits both children before processing the current index i.

Deletion (delete)

- Locates the element's index in the array.
- Implements a strict structural constraint: A node can only be deleted if it has no children (i.e., both its left and right child positions are either out of bounds or None). If children exist, it prompts the user to delete the children first to prevent breaking sub-tree linkages.

3. Interactive Menu Loop

The while True loop provides a continuous command-line menu interface, capturing user choices (int(input)) and dispatching them to the appropriate BinaryTree methods until the user selects option 10 to exit.

7b. Linked List,

PROGRAM:



Binary tree using Linkedlist.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Linkedlist.py (3.13.7)

File Edit Format Run Options Window Help

```
            print("Root inserted successfully.")
        else:
            print("Root already exists!")

    def find(self, value):
        if self.root is None:
            return None

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.data == value:
                return current

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        return None

    def insert_left(self, parent, value):
        node = self.find(parent)

        if node is None:
            print("Parent not found!")
        elif node.left is not None:
            print("Left child already exists!")
        else:
            node.left = Node(value)
            print("Left child inserted successfully.")

    def insert_right(self, parent, value):
        node = self.find(parent)

        if node is None:
            print("Parent not found!")
        elif node.right is not None:
            print("Right child already exists!")
        else:
            node.right = Node(value)
            print("Right child inserted successfully.")

    def search(self, value):
```

Binary tree using Linkedlist.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Linkedlist.py (3.13.7)

File Edit Format Run Options Window Help

```
        else:
            node.right = Node(value)
            print("Right child inserted successfully.")

    def search(self, value):
        if self.find(value):
            print("Element found.")
        else:
            print("Element not found.")

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    def display(self):
        if self.root is None:
            print("Tree is empty.")
            return

        print("Level-order display:")
        queue = [self.root]

        while queue:
            current = queue.pop(0)
            print(current.data, end=" ")

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        print()
```

Binary tree using Linkedlist.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Linkedlist.py (3.13.7)

File Edit Format Run Options Window Help

```
        print()

    def delete(self, value):
        if self.root is None:
            print("Tree is empty.")
            return

        if self.root.data == value:
            if self.root.left is None and self.root.right is None:
                self.root = None
                print("Element deleted successfully.")
            else:
                print("Delete its children first!")
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left:
                if current.left.data == value:
                    if (current.left.left is None and
                            current.left.right is None):
                        current.left = None
                        print("Element deleted successfully.")
                    else:
                        print("Delete its children first!")
                    return

                queue.append(current.left)

            if current.right:
                if current.right.data == value:
                    if (current.right.left is None and
                            current.right.right is None):
                        current.right = None
                        print("Element deleted successfully.")
                    else:
                        print("Delete its children first!")
                    return

                queue.append(current.right)

        print("Element not found!")
```

Binary tree using Linkedlist.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Linkedlist.py (3.13.7)

File Edit Format Run Options Window Help

```
        print("Element not found!")


bt = BinaryTree()

while True:
    print("\n--- BINARY TREE USING LINKED LIST ---")
    print("1. Insert Root")
    print("2. Insert Left Child")
    print("3. Insert Right Child")
    print("4. Search Element")
    print("5. Display Tree")
    print("6. Preorder Traversal")
    print("7. Inorder Traversal")
    print("8. Postorder Traversal")
    print("9. Delete Element")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter root value: "))
        bt.insert_root(value)

    elif choice == 2:
        parent = int(input("Enter parent value: "))
        value = int(input("Enter left child value: "))
        bt.insert_left(parent, value)

    elif choice == 3:
        parent = int(input("Enter parent value: "))
        value = int(input("Enter right child value: "))
        bt.insert_right(parent, value)

    elif choice == 4:
        value = int(input("Enter element to search: "))
        bt.search(value)

    elif choice == 5:
        bt.display()

    elif choice == 6:
        print("Preorder:", end=" ")
        bt.preorder(bt.root)
        print()

    elif choice == 7:

    elif choice == 7:
        print("Inorder:", end=" ")
        bt.inorder(bt.root)
        print()

    elif choice == 8:
        print("Postorder:", end=" ")
        bt.postorder(bt.root)
        print()

    elif choice == 9:
        value = int(input("Enter element to delete: "))
        bt.delete(value)

    elif choice == 10:
        print("Exiting program.")
        break

    else:
        print("Invalid choice! Try again.")
```

OUTPUT:

Positive Case;

IDLE Shell 3.13.7

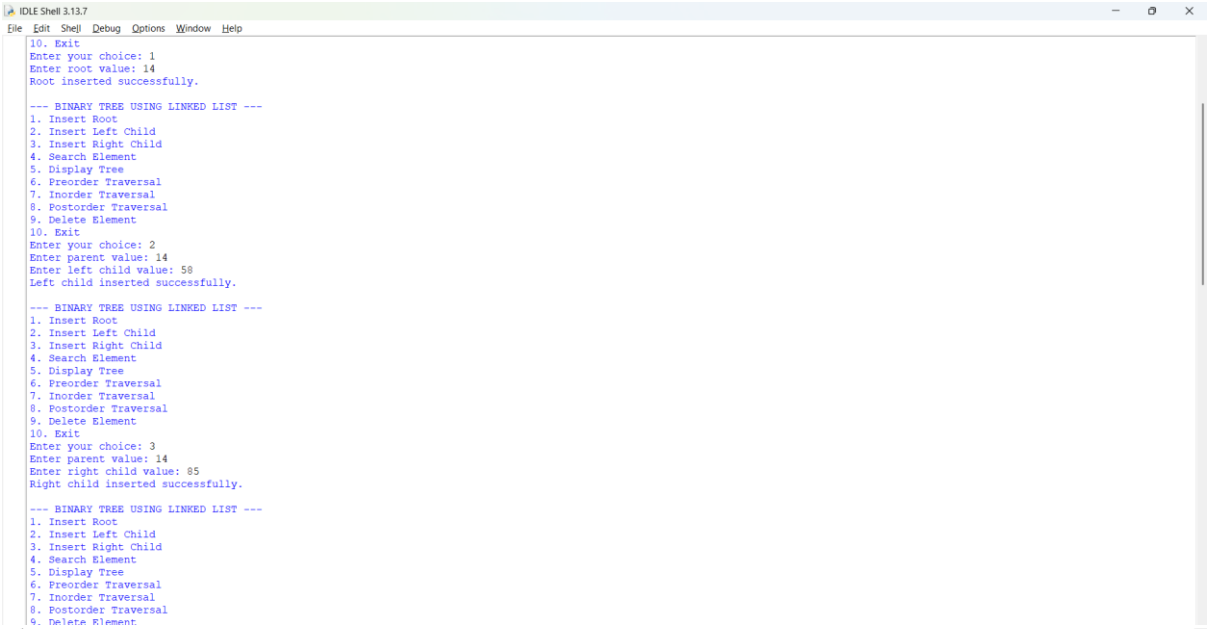
File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Linkedlist.py

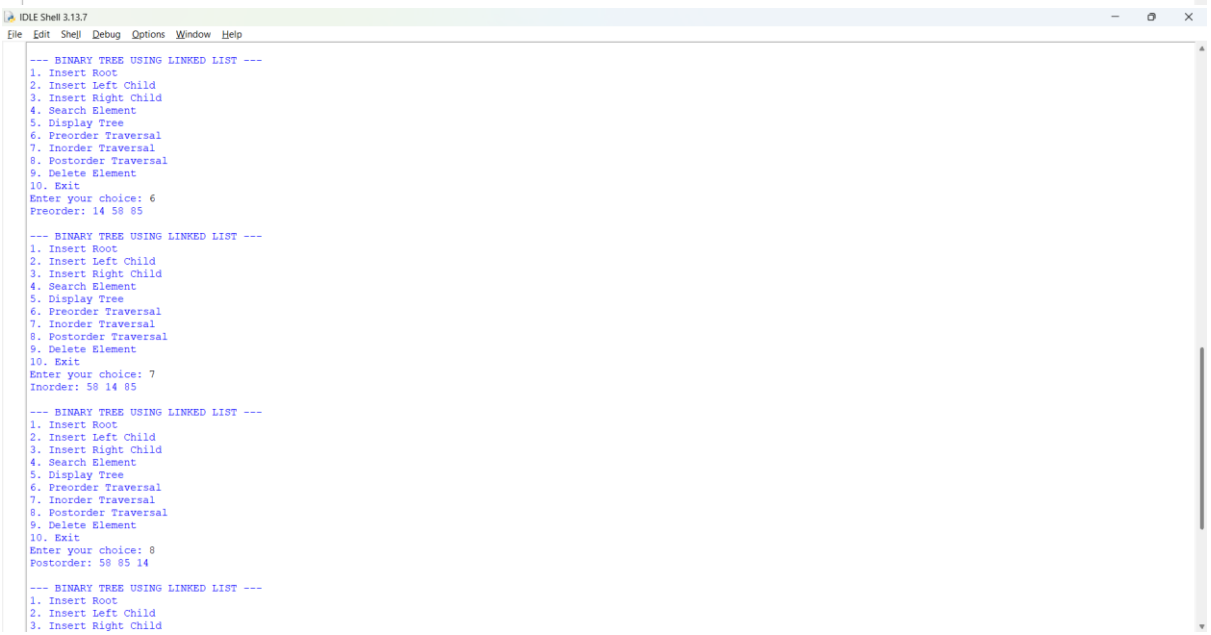
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 1
Enter root value: 14
Root inserted successfully.

--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 14
Enter left child value: 58
Left child inserted successfully.

--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
10. Exit
Enter your choice: 1
Enter root value: 14
Root inserted successfully.
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 14
Enter left child value: 58
Left child inserted successfully.
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 3
Enter parent value: 14
Enter right child value: 85
Right child inserted successfully.
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element


--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 3
Enter parent value: 14
Enter right child value: 85
Right child inserted successfully.
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 5
Level-order display:
14 58 85
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 4
Enter element to search: 14
Element found.


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 6
Preorder: 14 58 85
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 7
Inorder: 58 14 85
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 8
Postorder: 58 85 14
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 9
Enter element to delete: 14
Delete its children first!

--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 10
Exiting program.
>>>
```

Negative Case;

```
>>>
========== RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Linkedlist.py ==========
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 1
Enter root value: 10.5
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary tree using Linkedlist.py", line 169, in <module>
    value = int(input("Enter root value: "))
ValueError: invalid literal for int() with base 10: '10.5'
>>>
```

EXPLANATION:

1. Linked Representation:

Unlike the array-based approach where child and parent locations are computed using mathematical indices, a linked representation uses independent **Node objects** allocated dynamically in memory. Each node contains three fields:

- **data**: The value stored in the node.
- **left**: A reference (pointer) to the left child node.
- **right**: A reference (pointer) to the right child node.

Advantages:

- **Dynamic Size:** The tree can grow or shrink as needed without a predefined capacity limit.
- **Memory Efficient for Sparse Trees:** It only allocates memory for nodes that actually exist, avoiding wasted space.

Disadvantages:

- **Extra Memory Overhead:** Pointers (left and right) require additional memory space per node.
- **Slower Lookups:** Finding a parent node or checking a value requires searching through the tree (often $O(N)$ time complexity) because nodes do not store references back to their parents.

2. Breakdown of Classes and Key Methods

The Node Class

Acts as the basic building block, creating an independent container for data and reference pointers initialized to None.

The BinaryTree Class & Key Mechanics

- **find(value) (Breadth-First Search / Level-Order Search):** Since there is no index arithmetic, finding a specific node requires traversing the tree. This method uses a **Queue** (implemented via a Python list) to perform a level-order traversal, visiting nodes layer by layer from top to bottom until the target value is found.
- **Insertion (insert_root, insert_left, insert_right):**
  - insert_root creates the root node if self.root is empty.
  - insert_left and insert_right first call find(parent) to locate the parent node reference. If found and the target child slot is empty (None), a new Node is attached.
- **Tree Traversals (preorder, inorder, postorder):** These are recursive Depth-First Search (DFS) algorithms that take a starting node:
  - **Preorder (Root, Left, Right):** Visits the current node, then recursively traverses left and right subtrees.
  - **Inorder (Left, Root, Right):** Recursively traverses the left subtree, visits the current node, then traverses the right subtree.

    - **Postorder (Left, Right, Root):** Recursively traverses both subtrees before visiting the current node.

- **Level-Order Display (display):** Uses a queue to print all node values level by level, providing an intuitive view of the tree's hierarchical structure.
- **Deletion (delete):** Locates the target node via queue traversal. Just like the array version, it enforces a structural safety rule: **a node can only be deleted if it is a leaf node** (both its left and right pointers are None). If valid, the parent's pointer to it is severed by setting it to None.

3. Interactive Menu Loop

The while True loop runs a continuous command-line interface that captures user selections, executes the corresponding linked-tree methods, and prints status updates until option 10 is chosen to break the loop.

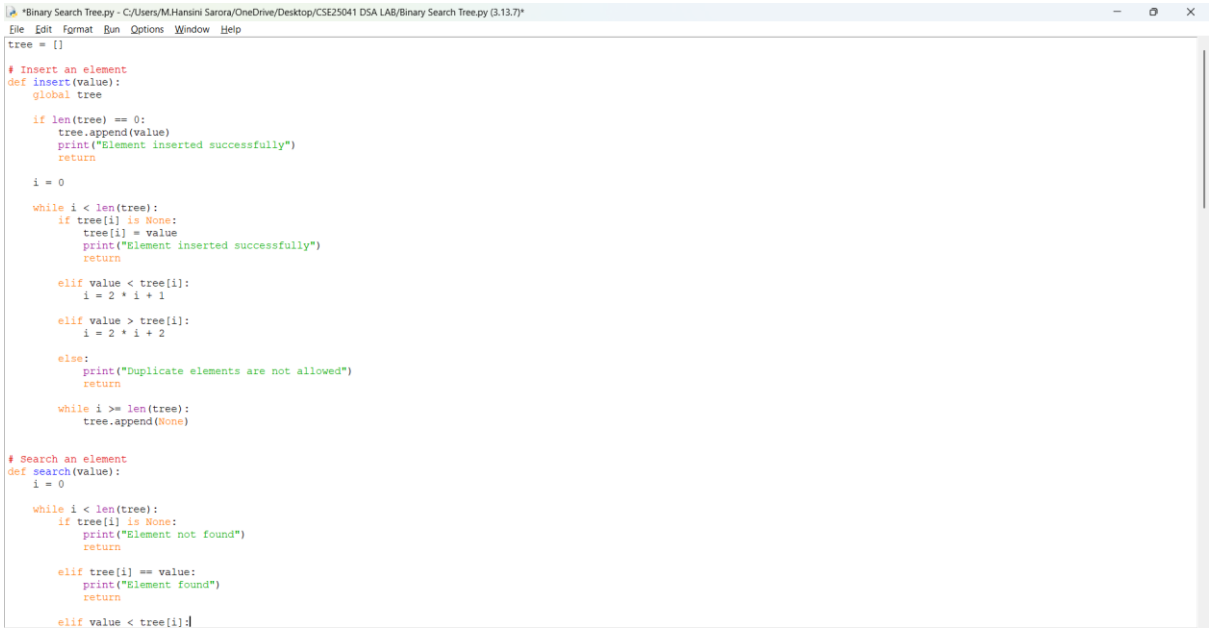
ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Index Error:** Attempting to access a position beyond array size.	Check array bounds before insertion or traversal.
2.	**Search Failure:** Requested key does not exist in the tree.	Inform the user and return an appropriate message.

WEEK – 8

8. AIM: Write a python program to implement Binary Search tree with its operation.

PROGRAM:



```
*Binary Search Tree.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary Search Tree.py (3.13.7)*
File  Edit  Format  Run  Options  Window  Help
tree = []

# Insert an element
def insert(value):
    global tree

    if len(tree) == 0:
        tree.append(value)
        print("Element inserted successfully")
        return

    i = 0

    while i < len(tree):
        if tree[i] is None:
            tree[i] = value
            print("Element inserted successfully")
            return

        elif value < tree[i]:
            i = 2 * i + 1

        elif value > tree[i]:
            i = 2 * i + 2

        else:
            print("Duplicate elements are not allowed")
            return

        while i >= len(tree):
            tree.append(None)


# Search an element
def search(value):
    i = 0

    while i < len(tree):
        if tree[i] is None:
            print("Element not found")
            return

        elif tree[i] == value:
            print("Element found")
            return

        elif value < tree[i]:
```



```
*Binary Search Tree.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary Search Tree.py (3.13.7)*
File  Edit  Format  Run  Options  Window  Help
        elif value < tree[i]:
            i = 2 * i + 1

        else:
            i = 2 * i + 2

    print("Element not found")


# Inorder traversal
def inorder(i=0):
    if i < len(tree) and tree[i] is not None:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)


# Preorder traversal
def preorder(i=0):
    if i < len(tree) and tree[i] is not None:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)


# Postorder traversal
def postorder(i=0):
    if i < len(tree) and tree[i] is not None:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")


# Display the array
def display():
    if len(tree) == 0 or tree[0] is None:
        print("Tree is empty")
    else:
        print("Array representation:", tree)


# Delete an element
def delete(value):
    global tree

    def find index(value):
```

Binary Search Tree.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary Search Tree.py (3.13.7)

File Edit Format Run Options Window Help

```
    index = find_index(value)

    if index == -1:
        print("Element not found")
        return

    # Collect all elements except the one to be deleted
    elements = [
        tree[i] for i in range(len(tree))
        if tree[i] is not None and i != index
    ]

    tree = []

    # Rebuild the BST
    for element in elements:
        insert(element)

    print("Element deleted successfully")


# Main menu
while True:
    print("\n\n--- BINARY SEARCH TREE MENU ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter element to insert: "))
        insert(value)

    elif choice == 2:
        value = int(input("Enter element to search: "))
        search(value)

    elif choice == 3:
        value = int(input("Enter element to delete: "))
        delete(value)
```

Binary Search Tree.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary Search Tree.py (3.13.7)

File Edit Format Run Options Window Help

```
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter element to insert: "))
        insert(value)

    elif choice == 2:
        value = int(input("Enter element to search: "))
        search(value)

    elif choice == 3:
        value = int(input("Enter element to delete: "))
        delete(value)

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        inorder()
        print()

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        preorder()
        print()

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        postorder()
        print()

    elif choice == 7:
        display()

    elif choice == 8:
        print("Program terminated")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

Positive Case;

IDLE Shell 3.13.7

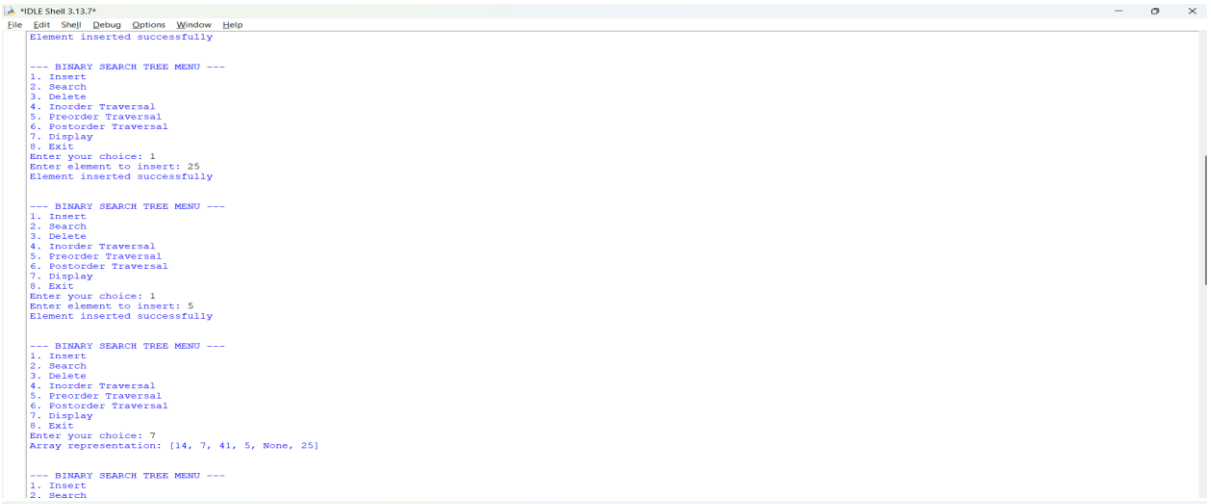
File Edit Shell Debug Options Window Help

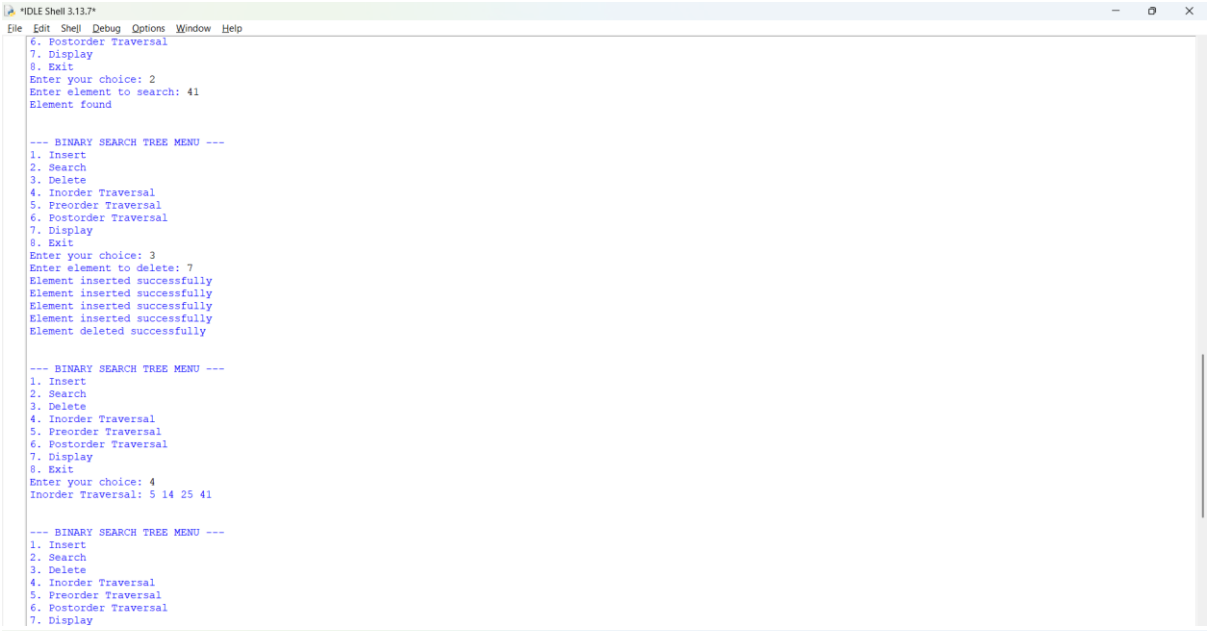
```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Binary Search Tree.py

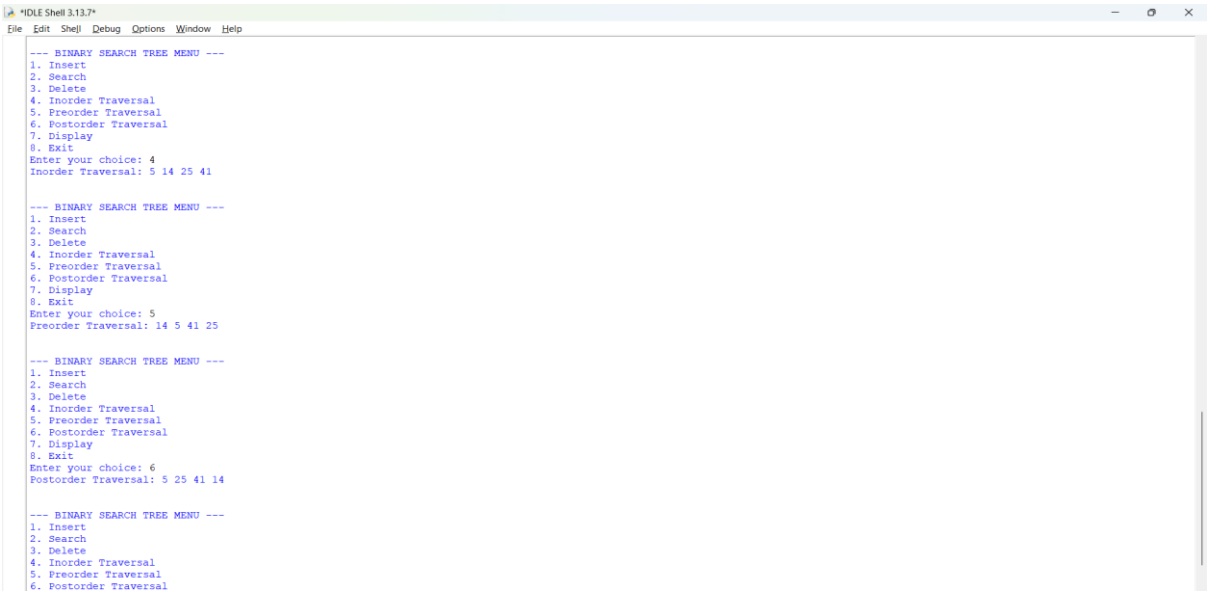

--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 14
Element inserted successfully


--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 41
Element inserted successfully


--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 7
Element inserted successfully
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Element inserted successfully
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 25
Element inserted successfully
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 5
Element inserted successfully
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 7
Array representation: [14, 7, 41, 5, None, 25]
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 2
Enter element to search: 41
Element found
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 3
Enter element to delete: 7
Element inserted successfully
Element inserted successfully
Element inserted successfully
Element inserted successfully
Element deleted successfully
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 4
Inorder Traversal: 5 14 25 41
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 4
Inorder Traversal: 5 14 25 41
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 5
Preorder Traversal: 14 5 41 25
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 6
Postorder Traversal: 5 25 41 14
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 6
Postorder Traversal: 5 25 41 14


--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 8
Program terminated
```

Negative Case;

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 25
Element inserted successfully


--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 25
Duplicate elements are not allowed
```

EXPLANATION:

1. Dynamic Array-Based BST:

Unlike the first fixed-size array implementation (which pre-allocated a static list of 100 elements), this version uses a **dynamic array** that automatically grows by appending None values whenever an insertion requires an index beyond the current list length.

It combines two core concepts:

1. **Binary Search Tree Property:** For any given node, all elements in its left subtree are smaller, and all elements in its right subtree are larger.
2. **Sequential Mapping:** Nodes are mapped to a linear list using the same index formulas ($2i + 1$ for left child, $2i + 2$ for right child), but the list expands dynamically on demand.

Advantages:

- No fixed size limits (unlike a static array).
- Maintains the arithmetic simplicity of array-based trees.

Disadvantages:

- **High Memory Overhead for Sparse Trees:** If the tree becomes heavily skewed (e.g., elements inserted in ascending order), the array will require massive padding with None values, consuming exponential amounts of memory relative to the number of nodes.

2. Breakdown of Key Operations

Insertion (insert)

- If the list is empty, the value becomes the root at index 0.
- Otherwise, it traverses the array using BST rules: moving to index 2 * i + 1 if the value is smaller, or 2 * i + 2 if it is larger.
- **Dynamic Expansion:** If the traversal index exceeds or equals the current length of the tree list (i >= len(tree)), the code enters a loop that appends None to the list until the target index becomes valid.

Search (search)

- Traverses the array using the same BST comparison logic.
- If it encounters a None slot or reaches the end of the array without finding the value, it concludes that the element is absent.

Traversals (inorder, preorder, postorder)

- Implemented recursively using array index formulas.
- Because it is a BST, an **Inorder traversal** naturally prints elements in **sorted ascending order**.

Deletion (delete)

Instead of handling complex pointer rewiring or structural rebalancing (like standard BST node deletion), this program uses a clever **re-insertion strategy**:

1. It finds the index of the target value.

2. If found, it extracts all valid (non-None) elements from the tree **except** the target value into a temporary list.
3. It resets the tree to an empty list (tree = []).
4. It loops through the remaining elements and re-inserts them one by one using the insert () function, effectively rebuilding a valid BST without the deleted node.

3. Interactive Menu Loop

The while True loop provides a continuous command-line interface, capturing user selections and executing the corresponding tree operations until option 8 is chosen to terminate the program.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Recursion Depth Error (Recursion Error):** Deep or skewed trees exceed Python's default recursion limit (usually 1000 calls) during recursive traversals (inorder, preorder, postorder).	Rewrite traversal functions iteratively using an explicit stack, or increase the recursion limit using sys.setrecursionlimit().
2.	**Stack Overflow in Traversals:** Recursive traversals on highly unbalanced trees may crash.	Implement iterative traversals using explicit stacks.

WEEK – 9

9. AIM: Write a python program to implement AVL trees with its operation.

PROGRAM:

AVL Trees.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/AVL Trees.py (3.13.7)

File Edit Format Run Options Window Help

```
class AVLTree:
    def __init__(self):
        self.keys = []
        self.left = []
        self.right = []
        self.height = []
        self.root = -1

    # Create a new node
    def create_node(self, key):
        index = len(self.keys)
        self.keys.append(key)
        self.left.append(-1)
        self.right.append(-1)
        self.height.append(1)
        return index

    # Get height
    def get_height(self, index):
        if index == -1:
            return 0
        return self.height[index]

    # Update height
    def update_height(self, index):
        self.height[index] = 1 + max(
            self.get_height(self.left[index]),
            self.get_height(self.right[index])
        )

    # Get balance factor
    def balance(self, index):
        if index == -1:
            return 0
        return (self.get_height(self.left[index])
                - self.get_height(self.right[index]))

    # Right rotation
    def right_rotate(self, y):
        x = self.left[y]
        t2 = self.right[x]

        self.right[x] = y
        self.left[y] = t2

        self.update_height(y)
        self.update_height(x)
```

AVL Trees.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/AVL Trees.py (3.13.7)

File Edit Format Run Options Window Help

```
        return x

    # Left rotation
    def left_rotate(self, x):
        y = self.right[x]
        t2 = self.left[y]

        self.left[y] = x
        self.right[x] = t2

        self.update_height(x)
        self.update_height(y)

        return y

    # Insert
    def insert_node(self, index, key):
        if index == -1:
            return self.create_node(key)

        if key < self.keys[index]:
            self.left[index] = self.insert_node(
                self.left[index], key
            )
        elif key > self.keys[index]:
            self.right[index] = self.insert_node(
                self.right[index], key
            )
        else:
            print("Duplicate values are not allowed!")
            return index

        self.update_height(index)
        bf = self.balance(index)

        # LL case
        if bf > 1 and key < self.keys[self.left[index]]:
            return self.right_rotate(index)

        # RR case
        if bf < -1 and key > self.keys[self.right[index]]:
            return self.left_rotate(index)

        # LR case
        if bf > 1 and key > self.keys[self.left[index]]:
            self.left[index] = self.left_rotate(
                self.left[index]
```

AVL Trees.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/AVL Trees.py (3.13.7)

File Edit Format Run Options Window Help

```
        if bf < -1 and key > self.keys[self.right[index]]:
            return self.left_rotate(index)

        # LR case
        if bf > 1 and key > self.keys[self.left[index]]:
            self.left[index] = self.left_rotate(
                self.left[index]
            )
            return self.right_rotate(index)

        # RL case
        if bf < -1 and key < self.keys[self.right[index]]:
            self.right[index] = self.right_rotate(
                self.right[index]
            )
            return self.left_rotate(index)

        return index

    # Search
    def search(self, index, key):
        if index == -1:
            return False

        if self.keys[index] == key:
            return True

        if key < self.keys[index]:
            return self.search(self.left[index], key)

        return self.search(self.right[index], key)

    # Find minimum node
    def min_node(self, index):
        while self.left[index] != -1:
            index = self.left[index]
        return index

    # Delete
    def delete_node(self, index, key):
        if index == -1:
            return -1

        if key < self.keys[index]:
            self.left[index] = self.delete_node(
                self.left[index], key
            )
```

AVL Trees.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/AVL Trees.py (3.13.7)

File Edit Format Run Options Window Help

```
        elif key > self.keys[index]:
            self.right[index] = self.delete_node(
                self.right[index], key
            )

        else:
            # No left child
            if self.left[index] == -1:
                return self.right[index]

            # No right child
            if self.right[index] == -1:
                return self.left[index]

            # Two children
            temp = self.min_node(self.right[index])
            self.keys[index] = self.keys[temp]

            self.right[index] = self.delete_node(
                self.right[index], self.keys[temp]
            )

        self.update_height(index)
        bf = self.balance(index)

        # LL case
        if bf > 1 and self.balance(self.left[index]) >= 0:
            return self.right_rotate(index)

        # LR case
        if bf > 1 and self.balance(self.left[index]) < 0:
            self.left[index] = self.left_rotate(
                self.left[index]
            )
            return self.right_rotate(index)

        # RR case
        if bf < -1 and self.balance(self.right[index]) <= 0:
            return self.left_rotate(index)

        # RL case
        if bf < -1 and self.balance(self.right[index]) > 0:
            self.right[index] = self.right_rotate(
                self.right[index]
            )
            return self.left_rotate(index)
```

AVL Trees.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/AVL Trees.py (3.13.7)

File Edit Format Run Options Window Help

```
                self.right[index]
            )
            return self.left_rotate(index)

        return index

    # Inorder traversal
    def inorder(self, index):
        if index != -1:
            self.inorder(self.left[index])
            print(self.keys[index], end=" ")
            self.inorder(self.right[index])

    # Preorder traversal
    def preorder(self, index):
        if index != -1:
            print(self.keys[index], end=" ")
            self.preorder(self.left[index])
            self.preorder(self.right[index])

    # Postorder traversal
    def postorder(self, index):
        if index != -1:
            self.postorder(self.left[index])
            self.postorder(self.right[index])
            print(self.keys[index], end=" ")

    # Display tree
    def display(self, index, level=0):
        if index != -1:
            self.display(self.right[index], level + 1)
            print("    " * level + str(self.keys[index]))
            self.display(self.left[index], level + 1)


# Main program
avl = AVLTree()

while True:
    print("\n\n===== AVL TREE USING ARRAYS =====")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display Tree")
```

```
AVL Trees.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/AVL Trees.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
    print("7. Display Tree")
    print("8. Exit")

    try:
        choice = int(input("Enter your choice: "))

        if choice == 1:
            key = int(input("Enter value to insert: "))
            avl.root = avl.insert_node(avl.root, key)
            print("Insertion completed.")

        elif choice == 2:
            key = int(input("Enter value to delete: "))

            if avl.search(avl.root, key):
                avl.root = avl.delete_node(avl.root, key)
                print("Node deleted successfully.")
            else:
                print("Value not found.")

        elif choice == 3:
            key = int(input("Enter value to search: "))

            if avl.search(avl.root, key):
                print("Value found.")
            else:
                print("Value not found.")

        elif choice == 4:
            print("Inorder:", end=" ")
            avl.inorder(avl.root)
            print()

        elif choice == 5:
            print("Preorder:", end=" ")
            avl.preorder(avl.root)
            print()

        elif choice == 6:
            print("Postorder:", end=" ")
            avl.postorder(avl.root)
            print()

        elif choice == 7:
            if avl.root == -1:
                print("Tree is empty.")
            else:
                avl.display(avl.root)

        elif choice == 8:
            print("Exiting program.")
            break

        else:
            print("Invalid choice!")

    except ValueError:
        print("Please enter a valid integer.")
```

OUTPUT:

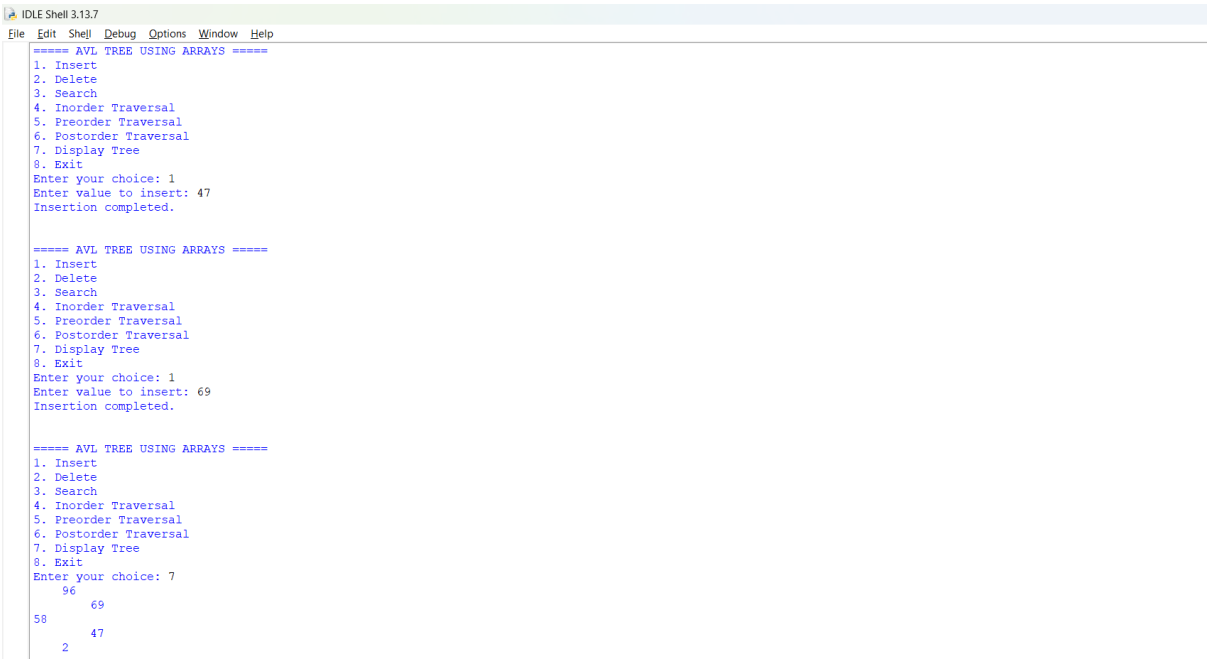
Positive Case;

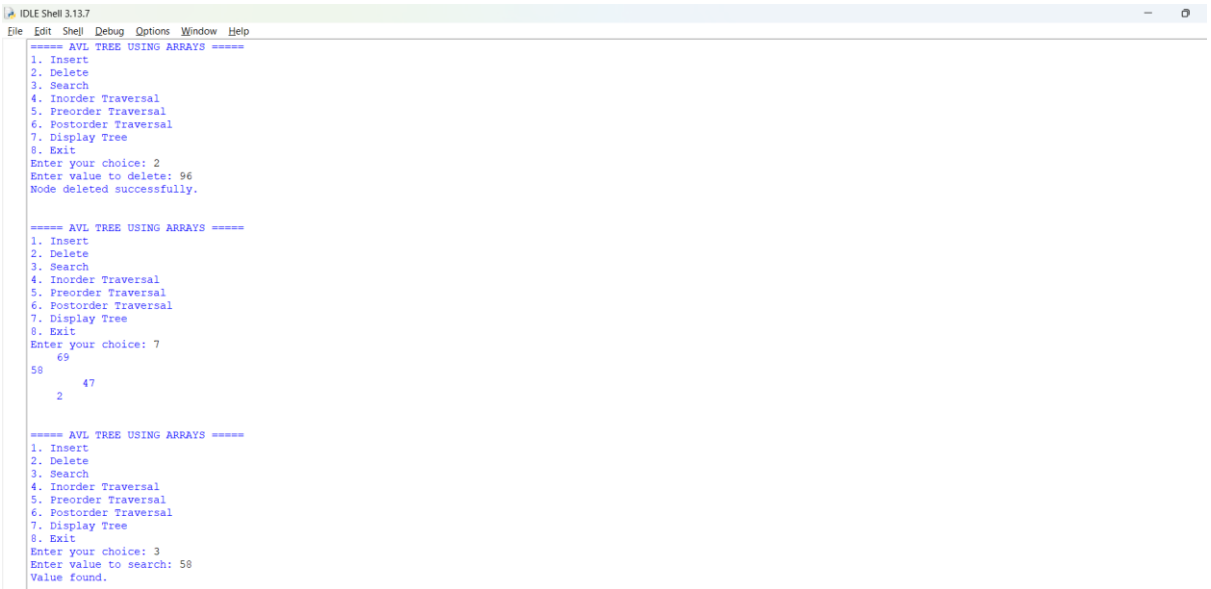
```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/AVL Trees.py

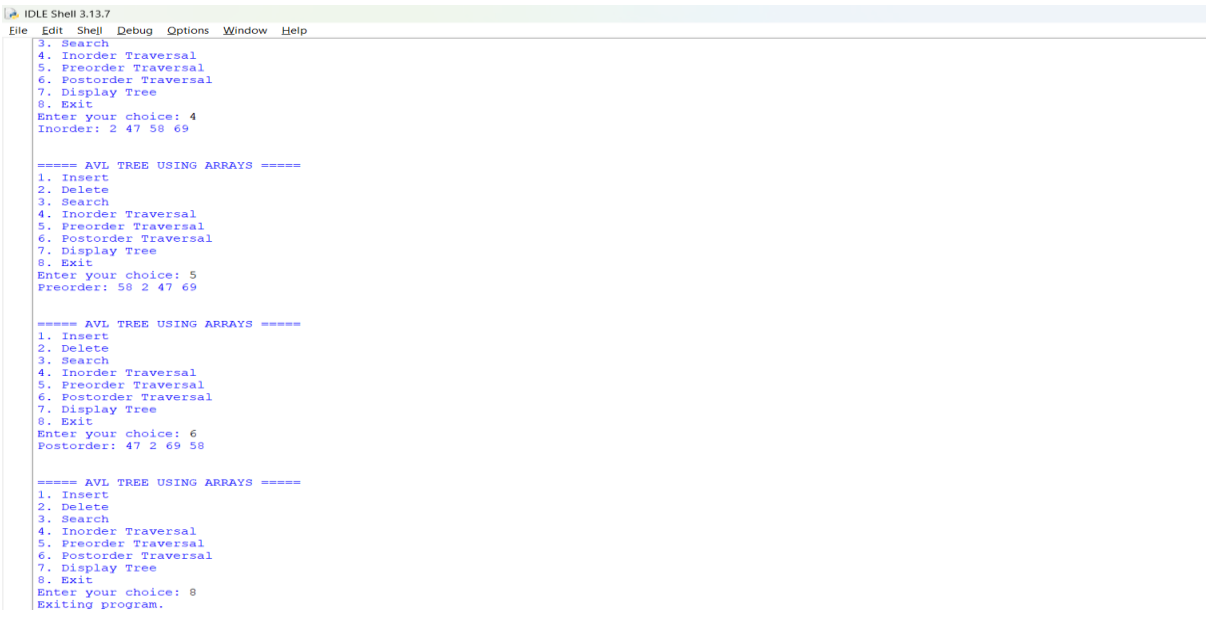

===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 58
Insertion completed.


===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 96
Insertion completed.

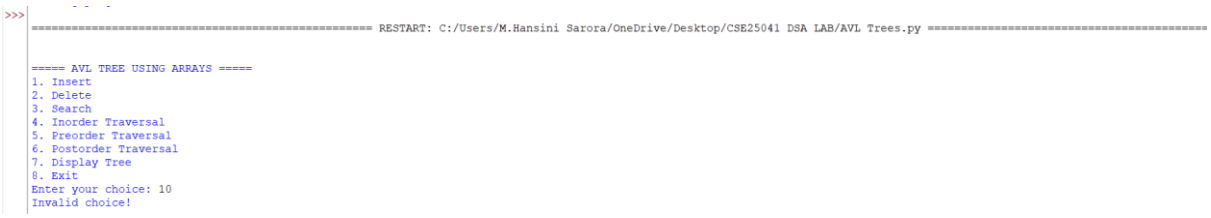

===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 2
Insertion completed.
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 47
Insertion completed.
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 69
Insertion completed.
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 7
96
69
58
47
2


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 2
Enter value to delete: 96
Node deleted successfully.
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 7
69
58
47
2
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 3
Enter value to search: 58
Value found.


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 4
Inorder: 2 47 58 69
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 5
Preorder: 58 2 47 69
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 6
Postorder: 47 2 69 58
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 8
Exiting program.

Negative Case;



EXPLANATION:

· The program implements an AVL tree using arrays in Python.

· The keys, left, right, and height arrays store node values, child indices, and heights.

· The create_node() function creates a new node and initializes its values.

· The get_height() function returns the height of a node.

· The update_height() function updates the height of a node.

· The balance() function calculates the balance factor of a node.

· The right_rotate() function performs a right rotation to balance the tree.

· The left_rotate() function performs a left rotation to balance the tree.

· The insert_node() function inserts new values and balances the tree when necessary.

· The program handles four imbalance cases: LL, RR, LR, and RL.

· The search() function checks whether a value exists in the tree.

· The min_node() function finds the minimum value in a subtree.

· The delete_node() function deletes a node and restores the tree's balance.

· The inorder() function displays values in ascending order.

· The preorder() function visits the root before its children.

· The postorder() function visits the root after its children.

· The display() function displays the tree structure.

· The main program provides a menu for insertion, deletion, searching, traversals, and displaying the tree.

· The try-except block handles invalid integer inputs.

· The program maintains the AVL tree's balance to make searching, insertion, and deletion efficient.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Root index not updated after rotation	Stored the returned root index in avl.root.
2.	Incorrect balance factor	Calculated the balance factor as left subtree height minus right subtree height.

WEEK – 10

10. AIM: Write a python program to implement hashing using Separating chaining, Linear and Quadratic probing

PROGRAM:

Hashing.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Hashing.py (3.13.7)

File Edit Format Run Options Window Help

```
SIZE = 11

chaining = [[] for _ in range(SIZE)]

linear = [None] * SIZE
quadratic = [None] * SIZE


def insert_chaining(key):
    index = key % SIZE
    chaining[index].append(key)
    print("Element inserted successfully.")


def search_chaining(key):
    index = key % SIZE
    if key in chaining[index]:
        print("Element found at index", index)
    else:
        print("Element not found.")


def display_chaining():
    print("\nHash Table using Separate Chaining:")
    for i in range(SIZE):
        print(i, ":", chaining[i])


def insert_linear(key):
    index = key % SIZE

    for i in range(SIZE):
        pos = (index + i) % SIZE

        if linear[pos] is None:
            linear[pos] = key
            print("Element inserted at index", pos)
            return

    print("Hash table is full.")


def search_linear(key):
    index = key % SIZE

    for i in range(SIZE):
        pos = (index + i) % SIZE
```

Hashing.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Hashing.py (3.13.7)

File Edit Format Run Options Window Help

```
        if linear[pos] is None:
            break

        if linear[pos] == key:
            print("Element found at index", pos)
            return

    print("Element not found.")


def display_linear():
    print("\nHash Table using Linear Probing:")
    for i in range(SIZE):
        print(i, ":", linear[i])


def insert_quadratic(key):
    index = key % SIZE

    for i in range(SIZE):
        pos = (index + i * i) % SIZE

        if quadratic[pos] is None:
            quadratic[pos] = key
            print("Element inserted at index", pos)
            return

    print("No empty position found.")


def search_quadratic(key):
    index = key % SIZE

    for i in range(SIZE):
        pos = (index + i * i) % SIZE

        if quadratic[pos] is None:
            break

        if quadratic[pos] == key:
            print("Element found at index", pos)
            return

    print("Element not found.")
```

```
Hashing.py - C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Hashing.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
def display_quadratic():
    print("\nHash Table using Quadratic Probing:")
    for i in range(SIZE):
        print(i, ":", quadratic[i])


# Main Menu
while True:
    print("\n===== HASHING TECHNIQUES =====")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 4:
        print("Program terminated.")
        break

    if choice not in [1, 2, 3]:
        print("Invalid choice.")
        continue

    while True:
        print("\n1. Insert")
        print("2. Search")
        print("3. Display")
        print("4. Back to Main Menu")

        operation = int(input("Enter your operation: "))

        if operation == 4:
            break

        if operation not in [1, 2, 3]:
            print("Invalid operation.")
            continue

        if operation in [1, 2]:
            key = int(input("Enter the element: "))

        if choice == 1:
            if operation == 1:
                insert_chaining(key)
            elif operation == 2:
                search_chaining(key)
          if operation in [1, 2]:
              key = int(input("Enter the element: "))

          if choice == 1:
              if operation == 1:
                  insert_chaining(key)
              elif operation == 2:
                  search_chaining(key)
              elif operation == 3:
                  display_chaining()

          elif choice == 2:
              if operation == 1:
                  insert_linear(key)
              elif operation == 2:
                  search_linear(key)
              elif operation == 3:
                  display_linear()

          elif choice == 3:
              if operation == 1:
                  insert_quadratic(key)
              elif operation == 2:
                  search_quadratic(key)
              elif operation == 3:
                  display_quadratic()
```

OUTPUT:

Positive Case;

```
>>>
    = RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Hashing.py

    ===== HASHING TECHNIQUES =====
    1. Separate Chaining
    2. Linear Probing
    3. Quadratic Probing
    4. Exit
    Enter your choice: 1

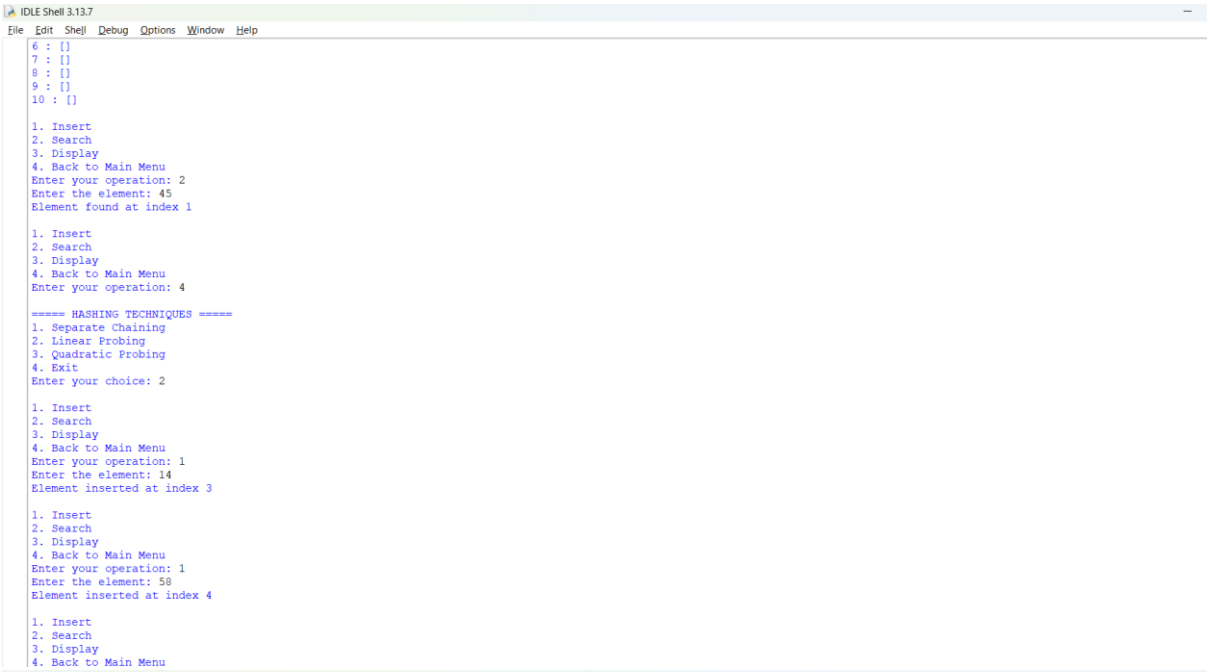
    1. Insert
    2. Search
    3. Display
    4. Back to Main Menu
    Enter your operation: 1
    Enter the element: 14
    Element inserted successfully.

    1. Insert
    2. Search
    3. Display
    4. Back to Main Menu
    Enter your operation: 1
    Enter the element: 45
    Element inserted successfully.

    1. Insert
    2. Search
    3. Display
    4. Back to Main Menu
    Enter your operation: 1
    Enter the element: 25
    Element inserted successfully.

    1. Insert
    2. Search
    3. Display
    4. Back to Main Menu
    Enter your operation: 3

    Hash Table using Separate Chaining:
    0 : []
    1 : [45]
    2 : []
    3 : [14, 25]
    4 : []
```



```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
6 : []
7 : []
8 : []
9 : []
10 : []

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 2
Enter the element: 45
Element found at index 1

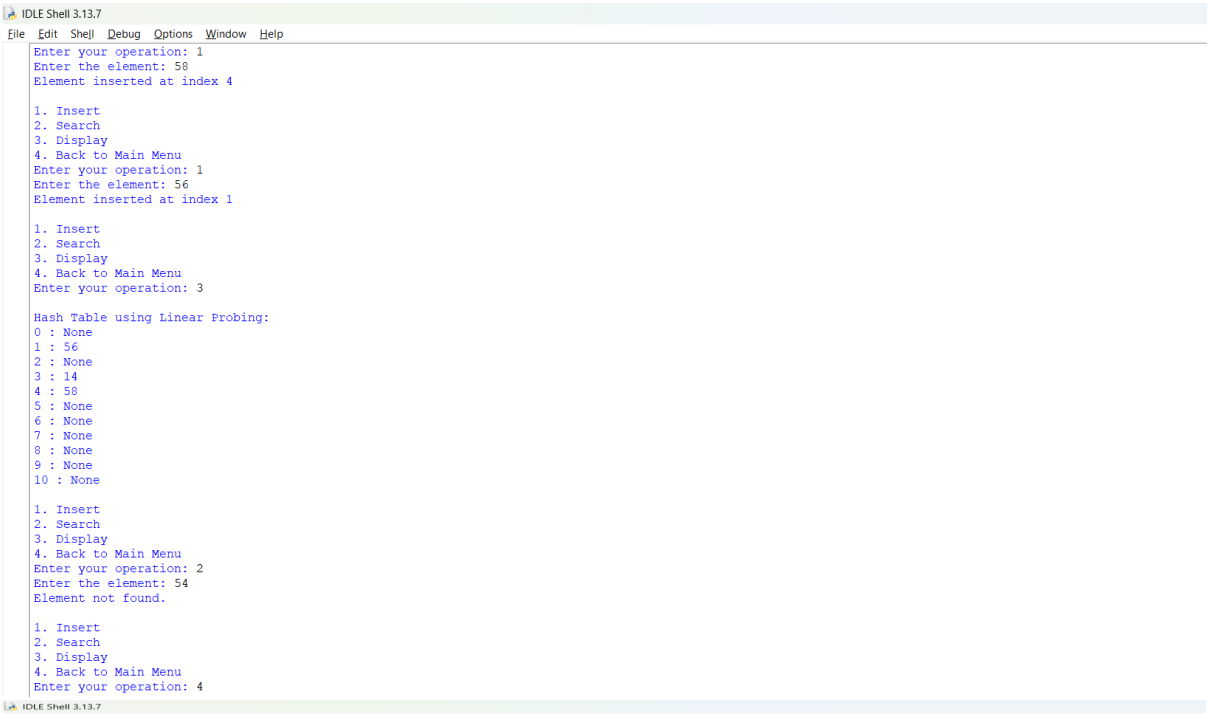
1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 4

===== HASHING TECHNIQUES =====
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 2

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 1
Enter the element: 14
Element inserted at index 3

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 1
Enter the element: 58
Element inserted at index 4

1. Insert
2. Search
3. Display
4. Back to Main Menu
```



```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
Enter your operation: 1
Enter the element: 58
Element inserted at index 4

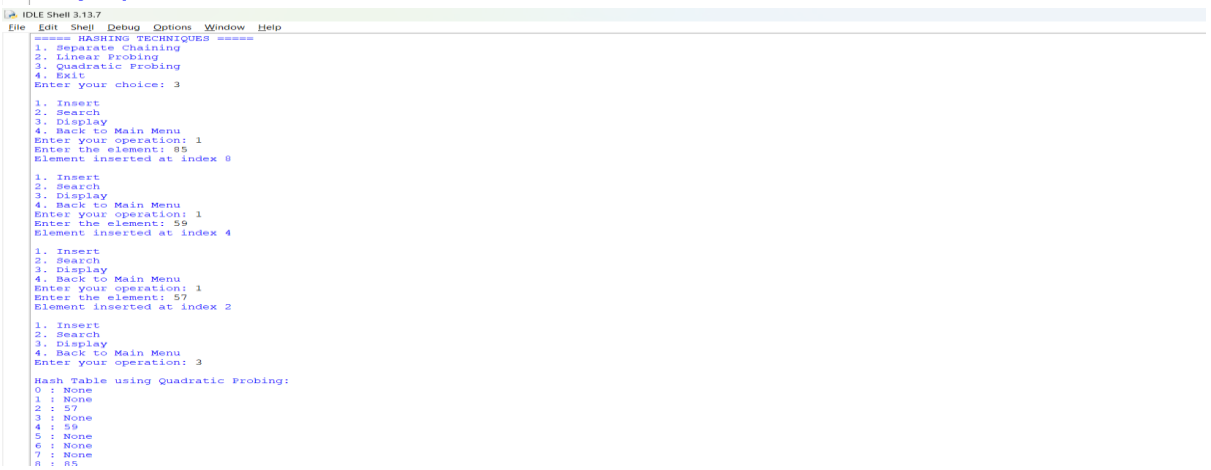
1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 1
Enter the element: 56
Element inserted at index 1

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 3

Hash Table using Linear Probing:
0 : None
1 : 56
2 : None
3 : 14
4 : 58
5 : None
6 : None
7 : None
8 : None
9 : None
10 : None

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 2
Enter the element: 54
Element not found.

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 4
```



```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
===== HASHING TECHNIQUES =====
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 3

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 1
Enter the element: 85
Element inserted at index 8

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 1
Enter the element: 59
Element inserted at index 4

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 1
Enter the element: 57
Element inserted at index 2

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 3

Hash Table using Quadratic Probing:
0 : None
1 : None
2 : 57
3 : None
4 : 59
5 : None
6 : None
7 : None
8 : 85
```

```
Hash Table using Quadratic Probing:
0 : None
1 : None
2 : 57
3 : None
4 : 59
5 : None
6 : None
7 : None
8 : 85
9 : None
10 : None

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 2
Enter the element: 85
Element found at index 8

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 4

===== HASHING TECHNIQUES =====
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 4
Program terminated.
```

Negative Case;

```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Hashing.py

===== HASHING TECHNIQUES =====
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 1

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: +
Traceback (most recent call last):
  File "C:/Users/M.Hansini Sarora/OneDrive/Desktop/CSE25041 DSA LAB/Hashing.py", line 125, in <module>
    operation = int(input("Enter your operation: "))
ValueError: invalid literal for int() with base 10: '+'
```

EXPLANATION:

· The program implements three hashing techniques: Separate Chaining, Linear Probing, and Quadratic Probing.

· The SIZE = 11 statement defines the size of the hash table as 11.

· The chaining list stores elements using separate chaining.

· The linear list stores elements using linear probing.

· The quadratic list stores elements using quadratic probing.

· The insert_chaining() function inserts an element into the corresponding index using the modulo operation.

· The search_chaining() function searches for an element in the corresponding chain.

· The display_chaining() function displays all the elements stored in the separate chaining hash table.

· The insert_linear() function inserts an element by checking consecutive positions when a collision occurs.

· The search_linear() function searches for an element using linear probing.

· The display_linear() function displays the elements in the linear probing hash table.

· The insert_quadratic() function resolves collisions by checking positions using the quadratic formula.

· The search_quadratic() function searches for an element using quadratic probing.

· The display_quadratic() function displays the elements in the quadratic probing hash table.

· The modulo operation key % SIZE calculates the initial hash index.

· The None value indicates an empty position in the linear and quadratic probing tables.

· The main menu allows the user to select a hashing technique or exit the program.

· The submenu allows the user to insert, search, display elements, or return to the main menu.

· The if-elif-else statements execute the selected operation.

· The program demonstrates how collisions are handled using three different hashing techniques.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Name Error:** Function or variable name typed incorrectly.	Verify spelling of function and variable names.
2.	**Indentation Error:** Improper indentation inside loops or functions.	Followed consistent indentation.
3.	**Clustering Problem in Linear Probing:** Consecutive occupied slots form clusters, degrading performance.	Use Quadratic Probing or Double Hashing to reduce clustering.